

Universidade de Vigo

ESCOLA SUPERIOR DE ENXEÑARÍA INFORMÁTICA

Memoria do Traballo de Fin de Grao que presenta

Pablo Freire Gullón

para a obtención do Título de
Graduado en Enxeñaría Informática

STOPP/START



Septiembre, 2026

Titor/a: Martín Pérez Pérez

Área de coñecemento: Linguaxes e Sistemas Informáticos

Departamento: Informática

Índice general

1	Introducción	1
1.1	Contexto del problema	1
1.2	Los criterios STOPP/START	1
1.3	Justificación del trabajo	1
1.4	Estado del arte	1
1.5	Cómo se aborda el problema	2
1.6	Integración de trabajos previos	2
2	Objetivos	3
2.1	Objetivo principal	3
2.2	Objetivos específicos	3
2.3	Fuera del alcance del trabajo	4
3	Solución propuesta y metodología	5
3.1	Visión general de la solución	5
3.2	Metodología de desarrollo	5
3.2.1	Desarrollo dirigido por pruebas (TDD)	5
3.2.2	Control de versiones y flujo de trabajo	6
4	Planificación y seguimiento	7
4.1	Planificación inicial	7
4.2	Seguimiento: estimación retrospectiva de dedicación	7
4.3	Desviaciones y su justificación	8
4.4	Puntos críticos identificados	9
5	Arquitectura	10
5.1	Visión general	10
5.2	Estructura de directorios	11
5.3	Módulos y responsabilidades	11
5.4	Dependencias entre módulos	12
5.5	Pipeline de evaluación	13
5.6	Decisiones arquitectónicas	13

6	Tecnologías y productos de terceros	15
6.1	Lenguaje y framework	15
6.2	Bibliotecas de terceros	16
6.3	Herramientas de desarrollo	16
6.4	Fuentes de datos clínicos	17
7	Especificación y análisis de requisitos	18
7.1	Actores del sistema	18
7.2	Requisitos funcionales	18
7.2.1	Criterios de aceptación de los requisitos funcionales	19
7.3	Requisitos no funcionales	19
7.4	Casos de uso	20
7.5	Análisis	20
8	Diseño del software	22
8.1	Diseño estático	22
8.1.1	Modelo de dominio	22
8.1.2	Estructura de un criterio	23
8.2	Diseño dinámico	24
8.2.1	Ciclo de evaluación	24
8.2.2	Variante: exportar PDF	24
8.2.3	Ciclo de persistencia	24
8.3	Diseño de la interfaz	25
8.3.1	Iteraciones de diseño con el tutor	25
9	Gestión de datos e información	27
9.1	Datos manejados por el sistema	27
9.2	Modelo de datos de los criterios	28
9.3	Persistencia	29
9.4	Privacidad y protección de datos	30
9.5	Generación de informes	31
10	Pruebas	32
10.1	Estrategia de pruebas	32
10.2	Pruebas unitarias	32
10.3	Pruebas de integración	33
10.4	Pruebas de interfaz	33
10.5	Verificación de conformidad con la guía STOPP/START	34
10.6	Inventario estático y estado de ejecución	34

10.7 Amenazas a la validez	35
10.8 Errores detectados y corregidos	35
11 Manual de usuario	37
11.1 Requisitos mínimos	37
11.2 Puesta en marcha local para evaluación	37
11.3 Guía de uso	38
11.3.1 Caso ficticio reproducible	38
11.3.2 Pantalla de medicamentos	39
11.3.3 Pantalla de diagnósticos y analítica	40
11.3.4 Resultados y generación del informe	40
11.4 Resolución de errores	40
11.5 Preguntas frecuentes	41
12 Principales aportaciones	42
12.1 Cumplimiento de los objetivos	43
12.2 Aportaciones técnicas	43
12.3 Aportaciones al ámbito clínico	44
13 Conclusiones	45
13.1 Conclusiones técnicas	45
13.2 Conclusiones personales	46
14 Vías de trabajo futuro	47
14.1 Paso de datos del paciente en la interfaz	47
14.2 Generalización de variantes y dependencias diagnósticas	47
14.3 Integración con la historia clínica electrónica	47
14.4 Normalización con SNOMED CT y entrada en lenguaje natural	48
14.5 Revisión profesional, dudas clínicas y catálogo	48
14.6 Matriz de correspondencia con la guía	48
Referencias	49
A Comandos de operación	51
B Sistemas clínicos y recuento de criterios	52
C Trazabilidad entre requisitos y evidencia	53
D Cobertura demostrable del catálogo de criterios	55

E	Criterios de aceptación de los requisitos funcionales	57
F	Resolución de incidencias de uso	61
G	Detalle de errores detectados en el motor	62
H	Esquemas de interfaz	63
I	Diagramas de secuencia	65
J	Esquema de una regla del catálogo	67
K	Diagramas del cuerpo de la memoria	68

Índice de figuras

3.1	Ciclo rojo-verde-refactor aplicado en el desarrollo.	6
5.1	Dependencias lógicas: los pasos orquestan; el motor no depende de la UI; solo el servicio de intercambio JSON accede directamente al estado de sesión. Ese estado no es una base de datos.	13
7.1	Entidades del dominio, con el nombre de cada relación sobre la línea y las multiplicidades en los extremos. Crit pertenece al catálogo; el resto constituye el caso de sesión.	21
8.1	Modelo de dominio simplificado. Crit no se persiste con el caso: el motor lo produce al evaluar.	23
H.1	Pantalla de medicamentos: pestañas de sistema, lista de fármacos en dos cubos y panel de criterios con acciones PDF/JSON.	63
H.2	Pantalla de diagnósticos: pestaña «Otros» con el panel fijo de analítica y el mismo panel de criterios a la derecha.	63
H.3	Panel de resultados STOPP/START, agrupado por sistema, visible en ambas pestañas del flujo.	64
I.1	Secuencia implementada al marcar una medicación: el estado de sesión cambia, el motor evalúa y la UI actualiza criterios y relevancia. El cálculo auxiliar de excludes no está integrado en este flujo.	65
I.2	Secuencia de exportación a PDF: evaluación al vuelo y composición en el cliente.	66
K.1	Funcionamiento general de la solución: en cada pestaña el motor evalúa el caso y marca el criterio como activo o inactivo; el resultado final distingue avisos STOPP y START.	68
K.2	Cronograma del proyecto (marzo-agosto de 2026): barras claras, planificación inicial; barras oscuras, actividad reconstruida mediante Git hasta el 16 de agosto. La fase de pruebas se solapa con la implementación. La redacción no se presenta como cerrada en esa fecha.	69

- K.3 Arquitectura de cajas: la UI actualiza el estado de sesión; el motor evalúa el caso con catálogos y `criteria.json`; las salidas leen una instantánea. El estado vive en memoria; `localStorage` guarda una copia en el navegador, no en una base de datos. `HttpClient` obtiene los activos mediante HTTP GET. 69
- K.4 Diagrama de casos de uso. Las líneas continuas asocian al actor con cada objetivo. Las discontinuas marcan la relación `include`: evaluar y persistir se ejecutan siempre que cambia la captura, sin invocación aparte. 70

Índice de cuadros

4.1	Comparación entre el plan y la estimación retrospectiva de dedicación hasta el 16 de agosto de 2026. No es un registro de horas ni el cierre definitivo del proyecto.	8
5.1	Directorios relevantes de la aplicación.	11
6.1	Productos de terceros y versión resuelta en el fichero de bloqueo.	16
7.1	Requisitos funcionales (resumen). El enunciado completo está en la tabla E.1.	19
7.2	Requisitos no funcionales, referidos al modelo de calidad ISO/IEC 25010. .	20
8.1	Campos de <code>Crit</code> a nivel de tipo.	23
9.1	Parámetros de Labs capturados en la pestaña «Otros» de diagnósticos. . . .	28
9.2	Claves de <code>localStorage</code> vigentes.	29
10.1	Inventario estático de la batería (conteo de cláusulas <code>it</code> (en los <code>*.spec.ts</code>), sin resultado de ejecución actual. El script <code>audit-criteria.cjs</code> no forma parte de este total.	35
12.1	Correspondencia entre los objetivos específicos (capítulo 2) y las aportaciones del trabajo.	43
A.1	Comandos de instalación, ejecución, prueba y auditoría.	51
B.1	Reglas implementadas por sistema clínico (catálogo <code>criteria.json</code>). . . .	52
C.1	Matriz requisitos–evidencia del repositorio.	53
D.1	Matriz de cobertura demostrable del catálogo, sin equivalencia 190→218. .	55
E.1	Requisitos funcionales del sistema implementado.	57
E.2	Criterios de aceptación de los requisitos funcionales.	60
F.1	Comprobaciones ante incidencias de uso y exportación.	61
J.1	Campos de una entrada de <code>criteria.json</code>	67

Capítulo 1

Introducción

1.1 Contexto del problema

En las personas mayores de 65 años, la interacción entre medicamentos y su relación con enfermedades aumenta el riesgo de reacciones adversas [1]. Ante esta situación se emplean criterios de cribado como STOPP/START, descritos en la sección siguiente [2].

1.2 Los criterios STOPP/START

Los criterios STOPP/START (*Screening Tool of Older Persons' Prescriptions / Screening Tool to Alert to Right Treatment*) constituyen una herramienta de cribado clínico orientada a la revisión de la medicación en personas mayores de 65 años [2]. Su objetivo es detectar, por un lado, prescripciones potencialmente inapropiadas (STOPP) y, por otro, omisiones de tratamientos indicados (START).

La versión vigente es la tercera (STOPP/START v3), publicada en 2023, que recoge 133 criterios STOPP y 57 criterios START, hasta un total de 190 [2]. En la práctica clínica habitual, el profesional debe consultar este listado —habitualmente distribuido en formato PDF [3]— e interpretarlo manualmente sobre el caso del paciente.

1.3 Justificación del trabajo

La aplicación manual de STOPP/START v3 exige contrastar los criterios con la información disponible de cada paciente. Este TFG explora la automatización de parte de ese contraste mediante una herramienta de apoyo a la decisión, sin sustituir el juicio clínico. No se ha medido el tiempo ahorrado ni la utilidad en consulta; el enfoque concreto y sus límites se describen a continuación.

1.4 Estado del arte

La automatización de la revisión farmacológica en personas mayores no parte de cero. Una revisión sistemática de 16 estudios sobre sistemas informatizados encontró reducciones

de prescripciones potencialmente inapropiadas en distintos contextos, pero señaló que la heterogeneidad de las intervenciones y la evidencia limitada sobre resultados clínicos impiden atribuirles un beneficio asistencial general [4].

Entre los antecedentes que aplican STOPP/START está SENATOR, que generaba recomendaciones individualizadas para pacientes hospitalizados. Su ensayo multicéntrico no redujo las reacciones adversas y registró una adopción baja de las recomendaciones por los profesionales [5]. El proyecto OPERAM empleó STRIPA, una aplicación web de apoyo a una revisión estructurada realizada por personal médico y farmacéutico. La intervención redujo prescripción inapropiada, pero no mostró una reducción significativa de los ingresos relacionados con medicamentos [6].

Estos trabajos tienen un alcance mayor que este TFG: integran más datos clínicos, intervención profesional estructurada y evaluación con pacientes. También muestran que generar avisos no equivale a mejorar resultados. La aportación aquí es más acotada: un prototipo sin servidor, explicable y reproducible que formaliza parte de STOPP/START v3 y hace visibles sus reglas durante la captura manual. No se presenta como sustituto de esos sistemas ni como herramienta clínicamente validada.

1.5 Cómo se aborda el problema

Para abordar el problema descrito en la sección 1.3, se desarrolla una aplicación web que evalúa reglas derivadas de STOPP/START con los datos presentes en el caso. La interfaz permite introducir medicación, diagnósticos y determinados valores analíticos; la edad y el sexo solo pueden llegar mediante la importación de un caso JSON, y de ambos el catálogo actual solo consulta la edad. Los umbrales de dosis y duración no se evalúan automáticamente, sino que se muestran en el aviso para su confirmación manual. Por tanto, una regla puede evaluarse con las entradas disponibles, requerir esa confirmación o no ser evaluable desde la interfaz. No se afirma una correspondencia completa entre las reglas del catálogo y los 190 criterios oficiales. Además, se deshabilitan opciones que no pueden activar ningún criterio con la selección actual y se destacan las relaciones entre medicamentos y diagnósticos. El detalle técnico se desarrolla en los capítulos de arquitectura y diseño.

1.6 Integración de trabajos previos

Este TFG no integra trabajos ni materiales procedentes de prácticas u otras asignaturas del grado. Sí se aplican, de forma indirecta, conceptos adquiridos a lo largo de la carrera —por ejemplo, ingeniería del software o diseño de interfaces—, pero el sistema se ha concebido y desarrollado de manera específica para este trabajo.

Capítulo 2

Objetivos

2.1 Objetivo principal

Desarrollar una aplicación web de apoyo a la decisión clínica que integre un motor de inferencia basado en los criterios STOPP/START v3, de modo que el profesional sanitario pueda obtener avisos a partir de los datos disponibles en el caso, distinguiendo la evaluación automática de la información que requiere confirmación manual.

2.2 Objetivos específicos

OE1: Formalizar como reglas los criterios STOPP/START v3 representables con el modelo de datos de la aplicación.

Alcance: catálogo declarativo de reglas STOPP y START. Su número y granularidad no acreditan por sí solos la cobertura de los 190 criterios oficiales; los aspectos que dependen de datos no capturados se conservan, cuando procede, como texto para la valoración del profesional.

OE2: Implementar un motor de reglas que evalúe el caso del paciente frente a dichas reglas.

Alcance: a partir de medicación, diagnósticos, analítica y los datos importados que cada regla consulte, determinar qué avisos STOPP y START se activan. El catálogo actual solo consulta la edad entre los datos demográficos; el sexo se conserva en el caso, pero no interviene en la lógica. No incluye integración con historias clínicas electrónicas externas ni inferencia de datos ausentes.

OE3: Diseñar un flujo de captura guiado de datos clínicos relevantes.

Alcance: interfaz para seleccionar medicamentos y diagnósticos e introducir los valores analíticos implementados. La interfaz no captura edad, sexo, dosis ni duración; edad y sexo solo pueden proceder de un JSON importado.

OE4: Incorporar ayudas de interfaz para guiar la revisión.

Alcance: deshabilitar opciones irrelevantes según la selección actual y destacar relaciones entre fármacos y diagnósticos; no incluye recomendaciones terapéuticas libres fuera de STOPP/START.

OE5: Presentar los resultados de forma clara y accionable.

Alcance: visualización de criterios activados (STOPP/START), agrupados de manera comprensible para la revisión clínica.

OE6: Permitir la generación de un informe y la reutilización del caso.

Alcance: exportación de informe (p. ej. PDF) y guardado/carga del caso (p. ej. JSON / persistencia local), sin almacenamiento en servidor.

OE7: Verificar el sistema mediante pruebas automáticas y contraste con la guía oficial.

Alcance: inventario de tests del motor y de la aplicación y comprobación de conformidad de las reglas con la interpretación realizada de la guía. No constituye un ensayo clínico, una revisión independiente completa, una validación con pacientes ni una evaluación formal de usabilidad.

2.3 Fuera del alcance del trabajo

El presente trabajo se centra en evaluar las reglas STOPP/START v3 que pueden resolverse con la información disponible en el caso. Según la regla, esa información procede de la interfaz o de un JSON importado; los umbrales de dosis y duración quedan para confirmación manual, y una regla que dependa de datos no disponibles no es evaluable por esa vía. Quedan deliberadamente fuera del alcance, entre otros, los siguientes aspectos:

- **Integración con la historia clínica electrónica.** La aplicación no se conecta a sistemas hospitalarios ni de atención primaria. En una evolución natural del sistema, la carga desde la historia clínica electrónica permitiría preseleccionar diagnósticos y medicaciones, reduciendo aún más el tiempo de uso en consulta.
- **Sustitución del juicio clínico.** La herramienta actúa como apoyo a la decisión: muestra avisos STOPP y START aplicables al caso, pero la interpretación y la decisión terapéutica final corresponden siempre al profesional sanitario.
- **Catálogo clínico acotado.** Medicamentos y diagnósticos disponibles son los modelados en la aplicación. No se pretende cubrir la totalidad del arsenal terapéutico ni todos los posibles códigos diagnósticos.
- **Arquitectura sin servidor.** El sistema opera íntegramente en el navegador, sin backend ni almacenamiento centralizado de pacientes, lo que simplifica la privacidad en el contexto del TFG pero limita escenarios multiusuario o de despliegue institucional.
- **Verificación sin validación clínica experimental.** La comprobación del sistema se basa en pruebas automáticas y en el contraste con la guía oficial. No se realiza una revisión independiente completa con perfil clínico, ni un ensayo, ni un estudio de impacto asistencial sobre pacientes reales, ni una evaluación formal de usabilidad.

Capítulo 3

Solución propuesta y metodología

3.1 Visión general de la solución

La solución es una aplicación web de apoyo a la decisión clínica cuyo núcleo es un motor de evaluación: las reglas STOPP/START v3 se mantienen aparte del mecanismo que las aplica, de modo que los criterios pueden actualizarse sin reescribir la lógica de la aplicación. El detalle interno del motor se desarrolla en los capítulos de arquitectura y diseño.

El profesional recorre un flujo en dos pestañas. En medicamentos selecciona los fármacos del paciente; el motor evalúa el caso y marca cada criterio como activo o inactivo. En diagnóstico se completa la comorbilidad y el motor vuelve a evaluar, con la misma distinción entre criterio activo e inactivo. El resultado final agrupa los avisos STOPP (prescripción potencialmente inapropiada) y START (posible omisión de tratamiento indicado). La figura K.1 del apéndice K resume este recorrido. La exportación e importación del caso se describen en capítulos posteriores.

3.2 Metodología de desarrollo

El desarrollo ha sido iterativo e incremental: cada cambio entra en incrementos pequeños y verificables, y se contrasta en rondas de revisión antes de darlo por cerrado. Esta forma de trabajar se justifica por el dominio del TFG. Un criterio mal codificado puede silenciar un aviso STOPP o START; conviene detectar ese fallo en el mismo incremento en que se introduce, contrastando el comportamiento con la guía oficial [2], [3].

En la práctica, el trabajo se organizó en rondas temáticas: revisión sistemática de los criterios STOPP y START por sistemas, una ronda de corrección del motor y los datos clínicos, y otra de servicios e interfaz. El detalle de planificación, horas y desviaciones se recoge en el capítulo 4; la batería de pruebas, en el capítulo 10.

3.2.1 Desarrollo dirigido por pruebas (TDD)

La implementación sigue Test-Driven Development [7]: antes de escribir código de producción se formula una prueba que falla (rojo); a continuación se escribe la implementación mínima que la hace pasar (verde); después se refactoriza con la suite en verde. El ciclo se

repite en incrementos pequeños, de modo que el sistema permanece en un estado ejecutable y comprobable.

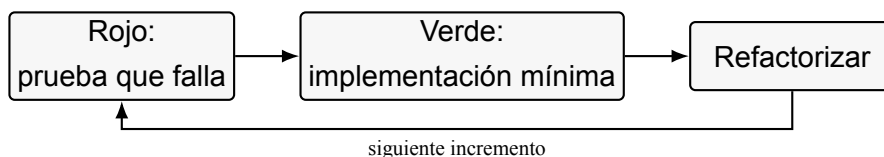


Figura 3.1: Ciclo rojo-verde-refactor aplicado en el desarrollo.

En este proyecto TDD no es solo una costumbre de código: es la defensa principal frente al falso negativo. Si un criterio no salta cuando la guía dice que debe saltar, el profesional no ve el aviso y el paciente puede permanecer con una prescripción inapropiada o sin un tratamiento indicado. Una prueba que describe el caso clínico (medicación, diagnósticos y, cuando aplica, constantes) y afirma si el criterio debe activarse o no convierte esa expectativa en una especificación ejecutable. El mismo esquema cubre el caso contrario —el criterio no debe disparar— para limitar los falsos positivos, que erosionarían la confianza en la herramienta.

Las pruebas se ejecutan con Karma y Jasmine sobre el código TypeScript de la aplicación. El criterio de avance de un incremento es que la nueva prueba pase y que el resto de la suite no se rompa. El inventario de pruebas y sus resultados se detallan en el capítulo 10.

3.2.2 Control de versiones y flujo de trabajo

El código se gestiona con Git. Los cambios de entidad —corrección de un conjunto de criterios, de un módulo del motor o de la interfaz— se desarrollan en ramas de trabajo y se fusionan a la línea principal cuando la suite de pruebas queda en verde.

Las ideas que alteran el alcance o la interfaz no se implementan directamente en la línea principal. Permanecen en cuarentena hasta que hay una decisión explícita de incorporarlas, posponerlas o descartarlas. Ejemplos de este filtro son la unificación de la orientación visual entre pestañas, dejada fuera del alcance fusionado, y el selector jerárquico de variantes diagnósticas, del que solo se incorporó la primera iteración (familia de hipertensión). De este modo se separa lo que ya está validado de lo que aún es hipótesis de diseño.

Capítulo 4

Planificación y seguimiento

4.1 Planificación inicial

El TFG se planificó para el periodo marzo–agosto de 2026, con una carga de 12 ECTS, equivalente a 300 horas de trabajo del estudiante. Esa cifra fijó el techo de la estimación inicial. El trabajo se descompuso en cinco fases, alineadas con la tabla de dedicación de la sección 4.2: análisis y documentación, diseño, implementación, pruebas y redacción de la memoria.

La fase de análisis concentró el estudio de la guía STOPP/START v3 [2], [3], la delimitación del alcance y la identificación de lo que debía quedar fuera del producto. El diseño cubrió el flujo de captura del caso, la organización de la interfaz y el modelo de las reglas, sin congelar de antemano todos los detalles de presentación. La implementación absorbió la mayor parte de las 300 horas, por ser la fase en la que el catálogo, el motor y la interfaz tienen que coexistir en un estado ejecutable. Las pruebas se planificaron en solapamiento deliberado con la implementación, de modo que cada incremento pudiera cerrarse con evidencia de comportamiento y no al final del calendario. La memoria se reservó para el tramo final, una vez el sistema fuera estable.

El solapamiento entre fases no es un accidente de calendario: el dominio clínico hace poco realista un diseño en cascada cerrado. Un criterio mal entendido en análisis reaparece en implementación; un cambio de interfaz pedido en una ronda de revisión obliga a retocar diseño y código a la vez. La figura K.2 del apéndice K muestra el cronograma previsto (barras claras) y la actividad reconstruida hasta el 16 de agosto de 2026 (barras oscuras). Esta última procede del histórico de Git y no equivale a un registro horario. El método de trabajo que justifica ese solapamiento se describe en el capítulo 3; este capítulo se limita al calendario y a la dedicación.

4.2 Seguimiento: estimación retrospectiva de dedicación

No se mantuvo un parte diario de tiempos. La tabla 4.1 ofrece, por tanto, una **estimación retrospectiva**, no horas reales medidas. Se reconstruyó a partir de la densidad y naturaleza de los cambios del histórico de Git y se contrastó con la carga prevista de 12 ECTS (300 h). Git acredita actividad, pero no la duración de cada tarea; las cifras deben leerse como apro-

ximaciones. Marzo corresponde al arranque del repositorio; abril, a la primera reunión de seguimiento; mayo, al esqueleto de la aplicación; junio, a la interfaz y a las pruebas de cada incremento; julio, a la auditoría del catálogo STOPP/START; y agosto, a la memoria.

El corte documental es el 16 de agosto de 2026. En esa fecha la redacción seguía abierta, por lo que ni su fila ni el total constituyen el cierre definitivo del proyecto.

Fase	Plan (h)	Est. retrospectiva (h)	Diferencia estimada
Análisis y documentación	40	~ 35	~ -5
Diseño	45	~ 62	~ +17
Implementación	120	~ 148	~ +28
Pruebas	50	~ 72	~ +22
Redacción de la memoria (al corte)	45	~ 28	~ -17
Total al corte	300	~ 345	~ +45

Cuadro 4.1: Comparación entre el plan y la estimación retrospectiva de dedicación hasta el 16 de agosto de 2026. No es un registro de horas ni el cierre definitivo del proyecto.

La estimación acumulada al corte es de aproximadamente 345 h, unas 45 h por encima de la previsión de 300 h. La diferencia estimada se concentra en diseño, implementación y pruebas. La cifra de memoria queda por debajo del plan porque esa fase aún no había terminado.

4.3 Desviaciones y su justificación

Tres hechos explican el grueso de las desviaciones positivas. Ninguno de ellos era un error de cómputo puntual: alteraron el calendario de más de una fase a la vez.

La primera causa es el diseño de la interfaz. En junio y julio se sucedieron rondas de revisión con el tutor informático que no estaban dimensionadas en el Gantt inicial como un bloque de iteración, sino como un diseño relativamente estable a partir de mayo. Cada ronda exigió reabrir decisiones de presentación y de flujo ya implementadas, de modo que el diseño se extendió hasta julio y arrastró retrabajo de implementación. Esa es la principal explicación de las aproximadamente 17 h adicionales atribuidas a diseño. El detalle de las iteraciones de interfaz corresponde al capítulo 8.

La segunda causa es la auditoría del catálogo STOPP/START, concentrada en julio. El contraste sistemático con la guía oficial detectó criterios inventados o redundantes en la sección B de STOPP y obligó a una revisión completa de STOPP y de START, no a un arreglo local. El trabajo recayó a la vez sobre implementación (corregir y completar reglas) y sobre pruebas (fijar el comportamiento esperado frente al texto de la guía). Es la contribución mayor a la diferencia estimada en implementación y a una parte de la estimada en pruebas. Los

fallos concretos hallados en el catálogo se recogen en el capítulo 10; aquí basta constatar que una auditoría de ese alcance no estaba prevista como un mes entero de calendario.

La tercera causa es el mantenimiento de una batería de pruebas junto con guardas sobre el catálogo a lo largo de toda la implementación, no solo al final. Esa disciplina —descrita en el capítulo 3— alarga cada incremento, explica el resto de la diferencia estimada en pruebas y sitúa el inicio observado de esa fase en mayo (figura K.2). El inventario pertenece al capítulo 10.

Las diferencias negativas tienen una explicación simétrica. El análisis se estima por debajo del plan porque el dominio venía acotado por una guía publicada. La memoria también queda por debajo en el corte porque julio se destinó a la auditoría y a las rondas de interfaz, y la redacción se desplazó a agosto, donde todavía continuaba el 16 de agosto.

4.4 Puntos críticos identificados

Tres puntos condicionaban al resto del calendario. Se identificaron en el análisis y se confirmaron en la ejecución.

El primero es la formalización de los 190 criterios de la guía como reglas evaluables. Hasta que un criterio no está expresado de forma unívoca, no hay aviso que mostrar ni prueba significativa. Un error aquí bloquea el catálogo y degrada cualquier fecha posterior. El volumen —133 STOPP y 57 START [2]— impedía tratarla como relleno final: tenía que avanzar sistema a sistema. La auditoría de julio lo confirmó: un conjunto pequeño de criterios mal codificados en una sección obligó a revisar el catálogo entero.

El segundo es la decisión temprana de un motor declarativo: las reglas viven aparte del mecanismo que las aplica. Esa elección fija el formato del catálogo y el orden de la implementación. Codificar cada criterio como rama de programa habría cambiado el calendario de corrección y habría reabierto código ante cada actualización de la guía. El resto de fases asumía que el catálogo podía crecer sin reescribir la aplicación. El capítulo 5 desarrolla esa decisión; aquí importa que era un hito de camino crítico.

El tercero es el riesgo de falso negativo clínico: un criterio que debería activarse y no lo hace. El profesional no ve el aviso, de modo que el fallo es más grave que un aviso espurio. No es una barra del Gantt, pero condiciona el cierre: no se podía terminar un sistema de criterios sin contrastarlo con la guía, ni comprimir las pruebas para recuperar retraso de interfaz. En julio se priorizó la auditoría frente a adelantar la memoria. El capítulo 3 justifica esa prioridad; el capítulo 10 documenta cómo se comprobó.

Capítulo 5

Arquitectura

Este capítulo describe las piezas del sistema y cómo se relacionan. El comportamiento interno de tipos y secuencias se desarrolla en el capítulo 8; el esquema de ficheros y la persistencia, en el capítulo 9. La visión de uso ya se adelantó en el capítulo 3.

5.1 Visión general

La aplicación es una SPA Angular 20 que se ejecuta por completo en el navegador [8]. No hay servidor de aplicación ni API propia: el conocimiento clínico viaja como activos estáticos y el caso permanece en el cliente. El profesional recorre dos rutas — /medicaciones y /diagnosticos— cuyos componentes actualizan el estado de sesión; el motor evalúa ese estado contra el catálogo y la interfaz muestra los avisos aplicables. Ese estado vive en memoria y se copia en el `localStorage` del navegador: no hay base de datos.

La figura K.3 resume el encaje. La capa de presentación (`MedsStepComponent`, `DiagnosisStepComponent` y los diálogos de guía, reinicio y opciones de visualización) no calcula reglas: muta el `CaseStoreService`, cuyos *signals* exponen el caso. El `CriteriaEngineService` carga `criteria.json` por HTTP desde `assets`, combina ese catálogo con los listados de fármacos y diagnósticos, y ofrece tres salidas: criterios que se cumplen, un cálculo interno de candidatos que activarían reglas con `excludes` e índice de relevancia por pestaña. El cálculo de candidatos está probado como capacidad del motor, pero la interfaz actual no lo presenta. Las salidas de caso —informe PDF y fichero JSON— las construyen `ReportService` y `CaseIoService`; los pasos entregan una instantánea al primero y el segundo accede al estado de sesión. El navegador aporta dos servicios de plataforma: `localStorage` para la copia del caso y `HttpClient` para los activos. La figura K.3 del apéndice K resume estas cajas y dependencias.

Los componentes son *standalone*: no hay `NgModule` de negocio. Los «módulos» de este capítulo son agrupaciones lógicas de ficheros con una responsabilidad, no módulos del compilador Angular.

5.2 Estructura de directorios

El código de aplicación vive bajo `src/app`. La tabla 5.1 recoge las carpetas que importan para entender el resto del capítulo.

Ruta	Contenido
<code>src/app/core</code>	Tipos de dominio, estado de sesión, motor, catálogos, informes e importación/exportación.
<code>src/app/core/data</code>	Listados y taxonomías de fármacos y diagnósticos; índice de relevancia y dependencias diagnóstico–medicación.
<code>src/app/core/services</code>	Motor de criterios y ayudas de prueba.
<code>src/app/steps</code>	Los dos pasos del flujo: medicación y diagnóstico.
<code>src/assets/data</code>	Catálogo declarativo <code>criteria.json</code> ; su esquema y recuento se documentan en el capítulo 9.

Cuadro 5.1: Directorios relevantes de la aplicación.

El arranque (`main.ts`, `app.config.ts`, `app.routes.ts`) monta un `router-outlet` y declara dos rutas más un comodín que redirige a medicaciones. No existe paso dedicado a los datos demográficos del paciente.

5.3 Módulos y responsabilidades

Dominio y estado de sesión. El fichero de tipos declara el agregado del caso —paciente, diagnósticos, medicaciones y analítica— y el tipo del criterio evaluable. Un único servicio de sesión es dueño de ese estado: una señal por campo, una instantánea inmutable del caso y operaciones de reinicio y carga. Los resultados del motor no se guardan: cada paso los recalcula. Al arrancar se recupera la copia guardada en el `localStorage` del navegador; cada cambio vuelve a escribirse ahí. Ese mecanismo no es una base de datos: es memoria de la sesión más una copia local del navegador. El detalle de claves y el tratamiento de datos personales se pospone al capítulo 9.

Catálogo clínico. Los módulos bajo `core/data` materializan el vocabulario que la UI y el motor comparten: fármacos con clases, diagnósticos normalizados, pestañas y grupos, y el mapa de dependencias que habilita o deshabilita un diagnóstico según la medicación. Sin este vocabulario común, una regla no podría referirse a «IECA» ni la interfaz agrupar casillas por sistema.

Motor. El servicio del motor carga el catálogo una vez, registra los operadores que JSON Logic no trae de serie [9] y evalúa el caso. No conoce Angular Material ni las rutas: recibe el caso y la lista de criterios, y devuelve los avisos aplicables, un mapa de fármacos que activarían advertencias de posible prescripción inapropiada y el índice de relevancia. Esa separación permite probar el motor sin montar la interfaz.

Pasos. Los dos componentes de paso son la única cara del flujo. Cada uno inyecta el estado de sesión, el motor y los servicios de salida; al iniciar pide la carga del catálogo y, a partir de ahí, señales derivadas y plantilla reaccionan a los cambios. Los diálogos —guía rápida, confirmación de reinicio y opciones de visualización— se abren desde la barra de cada paso, no desde el componente raíz.

Informes e intercambio. El servicio de informe compone el PDF a partir de los datos que le entrega el componente; no lee el estado de sesión. El servicio de intercambio sí lo hace: serializa el caso y valida el JSON versionado (v1.0) antes de reinyectarlo. Ninguno evalúa reglas por su cuenta. El texto plano para el portapapeles es una función pura auxiliar. La composición del PDF se detalla en el capítulo 9.

5.4 Dependencias entre módulos

La figura 5.1 muestra quién usa a quién. Las flechas van del cliente al proveedor. Los pasos dependen de los servicios de núcleo; el motor depende de HTTP y de los catálogos; el intercambio JSON lee el estado de sesión y el informe recibe sus datos desde los pasos. No hay ciclo: el estado de sesión no importa el motor ni los pasos.

El bloque central no es una base de datos ni la caché HTTP del navegador. Es el estado del caso en curso, en memoria, con una copia en el `localStorage` del origen de la aplicación: un almacén clave–valor del propio navegador, limitado a esa sesión y a ese sitio.

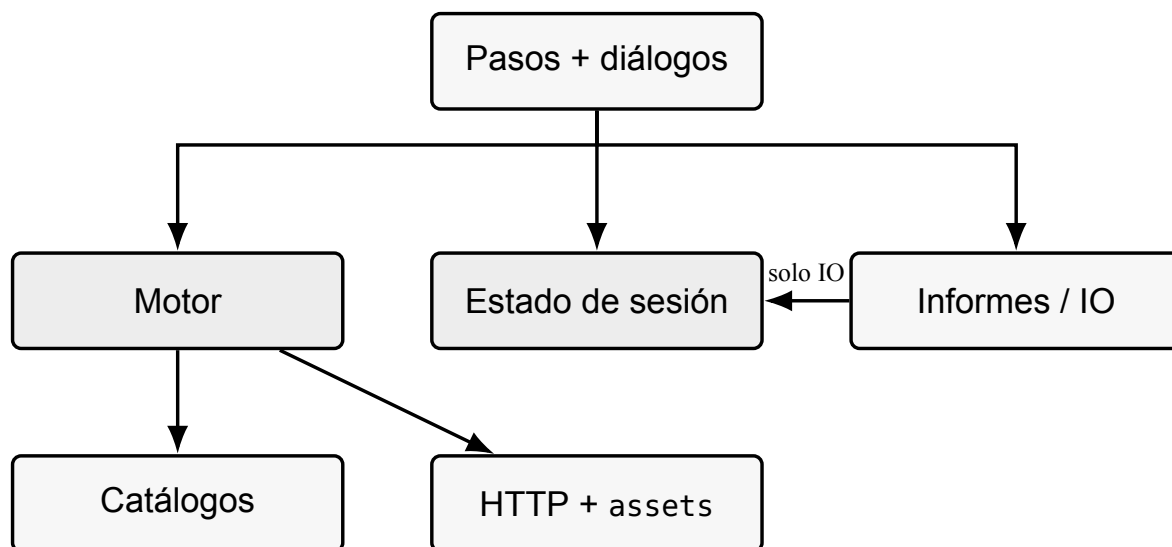


Figura 5.1: Dependencias lógicas: los pasos orquestan; el motor no depende de la UI; solo el servicio de intercambio JSON accede directamente al estado de sesión. Ese estado no es una base de datos.

5.5 Pipeline de evaluación

Al montar un paso, `loadCriteria` obtiene `criteria.json`, construye el índice de relevancia y el mapa de dependencias diagnóstico–medicación, y guarda la lista de criterios en una señal local. A partir de ese instante el pipeline es reactivo y no vuelve a pedir el fichero.

Cada vez que cambian medicación, diagnósticos o analítica, un *computed* del paso llama a `evaluate` con la instantánea `patientCase`. El motor normaliza el caso y filtra los criterios cuya lógica se cumple; la plantilla parte el resultado en grupos STOPP y START. El método `getExcludedMedications`, invocado en pruebas del motor pero no conectado a la plantilla, calcula qué fármacos del catálogo dispararían un criterio con campo `excludes` si se añadirían al caso —es el contrato técnico de advertencia preventiva del motor, no una prohibición clínica absoluta—. Por su parte, la señal `relevance` alimenta los dos cubos de cada pestaña (ítems propios e ítems relevantes de otros sistemas). El detalle de operadores y del árbol JSON se reserva al capítulo 8 y al capítulo 9.

5.6 Decisiones arquitectónicas

Las cuatro decisiones que siguen fijan la forma del sistema. En cada una se indica la alternativa descartada y el motivo.

SPA sin backend frente a aplicación con API. Una arquitectura clásica cliente–servidor habría colocado el motor y el almacenamiento en un servidor, con autenticación e integra-

ción futura a una historia clínica. Se descartó porque el TFG no contempla esa integración (capítulo 2) y porque un servidor propio introduciría un tratamiento de datos de salud que el alcance no justifica. La SPA deja el caso en el navegador y el motor junto a la interfaz: no hay canal de red con fichas de paciente. El coste es la ausencia de multiusuario y de despliegue institucional; ambos quedan fuera de alcance.

Reglas declarativas en JSON frente a *if/else* en TypeScript. Codificar cada criterio como ramas en el código acoplaría la guía al ciclo de compilación: un matiz clínico exigiría tocar funciones y adaptar pruebas específicas. JSON Logic permite expresar la regla como dato [9]: el catálogo se actualiza sin recompilar el motor, y una prueba puede cargar el mismo fichero que la aplicación. La contrapartida —operadores de dominio que el estándar no trae— se concentra en un único registro, no en una función por regla.

Signals frente a un estado basado en RxJS. El estado es local, síncrono y de un solo usuario. Un modelo basado en observables [10] habría añadido suscripciones, *subjects* y cancelación para un flujo que no espera eventos de red ni hilos concurrentes. Las señales de Angular, con *computed* y *effect*, cubren lectura, derivación y copia en *localStorage* sin esa maquinaria [8]. RxJS permanece en el proyecto solo donde el propio framework lo exige (por ejemplo, *HttpClient*).

Dos pasos frente a un asistente de cinco pantallas. Un asistente largo —paciente, analítica, medicación, diagnóstico, resultados— fragmentaría un acto que en consulta es único. Medicación y diagnóstico bastan para disparar la mayoría de los criterios; la analítica cabe en la pestaña «Otros» del segundo paso, porque varios avisos START dependen de constantes que ninguna casilla «activa». No hay paso de ficha demográfica ni pantalla final de resultados: los avisos viven en el propio paso, a la vista mientras se marca.

Capítulo 6

Tecnologías y productos de terceros

Este capítulo describe el lenguaje, el marco, las bibliotecas y las herramientas, y deja explícito qué no es de autoría propia: Angular y sus dependencias, las bibliotecas de reglas, validación y PDF, y la fuente clínica STOPP/START v3. La autoría del TFG se limita a la aplicación que las integra. La organización interna se trata en el capítulo 5.

6.1 Lenguaje y framework

El sistema está escrito en TypeScript 5.9 y se ejecuta como una aplicación de una sola página (SPA) sobre Angular 20 [8]. La elección de una SPA de cliente —sin servidor de aplicación— responde al alcance del TFG: el profesional introduce el caso, el motor lo evalúa en el navegador y el resultado no sale de esa sesión salvo que se exporte. Un backend habría exigido autenticación, almacenamiento remoto y un despliegue que el trabajo no contempla, como ya se delimita en el capítulo 2. Angular cubre ese escenario con enrutado, plantillas y un ciclo de vida de componentes suficiente para un flujo de dos pestañas, sin introducir una API propia.

TypeScript en modo estricto no es un adorno de estilo. El dominio maneja identificadores de fármacos, diagnósticos y criterios que deben conservarse de un extremo a otro; un tipado laxo permitiría mezclarlos sin que el compilador protestara. En la frontera del JSON importado el tipado estático no basta, y por eso se recurre a Zod, más abajo.

Angular 20 se eligió, además, por el modelo de componentes independientes (*standalone*) y por las señales (*signals*). Los componentes independientes declaran sus propias importaciones y evitan el andamiaje de módulos `NgModule`, desproporcionado para una sola aplicación. Las señales permiten expresar el estado del caso —medicación, diagnósticos, resultado de la evaluación— como valores que la plantilla consume de forma síncrona, sin convertir cada cambio en una tubería de observables. RxJS sigue presente porque Angular lo requiere; no se ha construido la interfaz alrededor de él. Material [11] aporta los componentes de pestañas, listas y controles usados en la interfaz. Su empleo no se toma como prueba de accesibilidad del producto completo, que requeriría una auditoría específica.

6.2 Bibliotecas de terceros

La tabla 6.1 resume los productos usados en tiempo de ejecución y el motivo de cada uno. Las versiones son las resueltas en `package-lock.json`, no solo los rangos del manifiesto.

Producto	Versión	Uso y justificación
Angular	20.3.4	SPA: componentes, enrutado y plantillas.
Angular Material	20.2.8	Pestañas, listas y campos de interfaz.
json-logic-js	2.0.5	Evalúa reglas JSON. Alternativa: criterios en código.
pdfmake	0.2.20	PDF declarativo en el navegador. Alternativa: jsPDF.
zod	4.4.3	Valida el JSON importado en tiempo de ejecución.
RxJS	7.8.2	Dependencia reactiva de Angular; no modela el estado.
zone.js	0.15.1	Detección de cambios por defecto de Angular.

Cuadro 6.1: Productos de terceros y versión resuelta en el fichero de bloqueo.

`json-logic-js` [9] evalúa las reglas del catálogo, serializadas como JSON. La alternativa —incrustar cada criterio en TypeScript— acopla el texto clínico al programa: corregir un aviso exigiría tocar código de producción. El evaluador JSON invierte esa dependencia; extender operadores se paga una vez, no por cada criterio.

`pdfmake` [12] genera el informe en el navegador. Se prefirió a `jsPDF` porque este construye la página de forma imperativa, mientras que `pdfmake` parte de un objeto de definición —bloques, tablas, estilos— más cercano a un informe de avisos STOPP y START.

`Zod` [13] valida el JSON importado antes de cargarlo: el compilador no ve ese fichero. No sustituye el tipado estático ni modela el catálogo interno. `RxJS` [10] y `zone.js` viajan con Angular —biblioteca reactiva y detección de cambios— y no son un diseño propio del autor.

6.3 Herramientas de desarrollo

El código se versiona con Git. El capítulo 3 describe el flujo de ramas; aquí Git es la herramienta de registro, la misma que sirve como indicio para la estimación retrospectiva del capítulo 4.

Angular CLI (`ng serve`, `ng test`, `ng build`) es el entorno de desarrollo y de construcción. No hay integración continua propia: la verificación se ejecuta en local con la CLI y el compilador de TypeScript, que comprueba los tipos. El proyecto no añade un linter como criterio de entrega más allá del andamiaje de Angular.

Las pruebas automáticas se ejecutan con Karma y Jasmine. Karma lanza el navegador; Jasmine expresa los casos. La estrategia, los conteos y la validación frente a la guía corresponden al capítulo 10.

6.4 Fuentes de datos clínicos

El contenido clínico no es de autoría propia. Los criterios implementados derivan de STOPP/START versión 3, publicados por O'Mahony y colaboradores [2]. Esa fuente define 133 criterios STOPP y 57 START (190 en total) y es el referente de corrección: un aviso de la aplicación debe corresponder a un criterio de la guía, no a una regla inventada.

La redacción en español sigue la traducción de la Gerencia Regional de Salud de Castilla y León (SACYL) [3]. El TFG no produce una nueva traducción ni una nueva versión de los criterios; una divergencia respecto de esas fuentes es un defecto del sistema, no una propuesta clínica. Los medicamentos y diagnósticos de la interfaz son el subconjunto modelado para evaluar las reglas de este trabajo: no cubren el vademécum completo ni todos los diagnósticos posibles. El enunciado de cada criterio pertenece a STOPP/START v3 y a su traducción; el TFG aporta la formalización ejecutable.

Capítulo 7

Especificación y análisis de requisitos

Los requisitos que siguen describen el sistema *tal como está implementado*. No se incluyen capacidades deliberadamente fuera de alcance (sección 2.3): integración con la historia clínica electrónica, paso dedicado de ficha demográfica ni sustitución del juicio clínico. Cada requisito funcional es verificable sobre la interfaz o sobre el resultado de la evaluación. Los criterios de aceptación observables se recogen en el apéndice E (tabla E.2); no presuponen que la prueba se haya ejecutado ni sustituyen la evidencia del capítulo 10.

7.1 Actores del sistema

El actor primario es el **profesional sanitario** que revisa la medicación de una persona mayor —geriatra, médico clínico o farmacéutico asistencial—. Interactúa con la aplicación en consulta: selecciona fármacos y diagnósticos, anota analítica cuando procede y examina los avisos STOPP y START. El sistema es de *apoyo a la decisión*: muestra criterios aplicables, pero no prescribe, no cierra el caso clínico y no sustituye la interpretación del profesional [2].

No hay actores secundarios de sistema (servidor, historia clínica, laboratorio externo). La persistencia y la evaluación ocurren en el navegador del propio profesional, según la visión del capítulo 3.

7.2 Requisitos funcionales

La tabla 7.1 identifica los requisitos funcionales con prioridad. La descripción completa, sin ambigüedad, está en la tabla E.1 del apéndice E. La prioridad *Alta* corresponde a capacidades sin las cuales el flujo de revisión no se completa; *Media*, a ayudas de captura, accesibilidad o reutilización del caso.

ID	Prioridad	Enunciado breve
RF01	Alta	Seleccionar y deseleccionar medicamentos por categoría.
RF02	Alta	Seleccionar diagnósticos, con variantes excluyentes y dependencias.
RF03	Alta	Capturar analítica y constantes en la pestaña «Otros».
RF04	Alta	Reevaluar el caso en el cliente tras cada cambio.
RF05	Baja	Calcular exclusiones preventivas (no expuesto en la UI).
RF06	Alta	Mostrar relevantes de otros sistemas en la pestaña activa.
RF07	Media	Marcar una pestaña vacía como revisada.
RF08	Alta	Agrupar avisos por tipo STOPP/START y por sistema.
RF09	Alta	Descargar el informe PDF del caso.
RF10	Media	Copiar los criterios aplicables como texto plano.
RF11	Alta	Exportar e importar el caso en JSON versionado.
RF12	Media	Reiniciar el caso tras confirmación.
RF13	Media	Persistir escala de fuente y orientación de pestañas.
RF14	Media	Abrir la guía rápida y el enlace al PDF oficial.
RF15	Media	Añadir un ítem «Otro» por grupo del catálogo.

Cuadro 7.1: Requisitos funcionales (resumen). El enunciado completo está en la tabla E.1.

7.2.1 Criterios de aceptación de los requisitos funcionales

Para evitar repetir cada descripción, la tabla E.2 del apéndice E agrupa requisitos que comparten procedimiento. La comprobación parte de un caso vacío salvo que el propio escenario requiera datos previos.

7.3 Requisitos no funcionales

La tabla 7.2 fija cinco condiciones de calidad, cada una con un criterio observable. Se toman como referencia las características de ISO/IEC 25010 [14]: usabilidad del flujo, ejecución en el cliente, adaptación visual, mantenibilidad de las reglas y tratamiento local del caso. No se incluye un tiempo máximo de respuesta porque no se realizó ninguna medición.

ID	Característica	Requisito	Criterio de aceptación
RNF01	Usabilidad	El flujo tendrá dos pasos (medicamentos y diagnósticos) y mostrará los avisos en la misma vista que la captura.	Se recorren las dos rutas y en ambas se puede seleccionar una entrada y observar sus avisos sin abrir una tercera pantalla.
RNF02	Eficiencia de desempeño	La evaluación de criterios se ejecutará en el cliente. Después de cargar los activos, un cambio del caso no requerirá una petición de red del motor.	Con el catálogo ya cargado, una prueba cambia una entrada y observa la actualización; la inspección de red no muestra una petición nueva asociada a evaluar el caso. No se evalúa latencia.
RNF03	Adaptabilidad visual	Existirán tres escalas de fuente (1, 1,15 y 1,3) aplicables a toda la interfaz, con persistencia de la elección.	Cada opción cambia la variable de escala global y el valor elegido permanece tras recargar. Este requisito no acredita por sí solo conformidad WCAG ni usabilidad clínica.
RNF04	Mantenibilidad	Las reglas clínicas residirán en un catálogo JSON independiente del código de la interfaz, de forma que un criterio pueda corregirse o añadirse sin reescribir el motor.	El motor carga una regla válida desde el catálogo o una prueba y la evalúa sin una rama específica para su identificador; los operadores de dominio siguen siendo código cuando la expresión los necesita.
RNF05	Seguridad / privacidad	El caso no se enviará a un servidor de aplicación; la persistencia será local y el reinicio permitirá retirar los valores clínicos del estado persistido (capítulo 9).	La inspección de red durante captura y evaluación no contiene el caso; tras confirmar el reinicio el estado restaurable es un caso vacío, aunque puedan persistir claves técnicas con valores vacíos. Un JSON de importación inválido se rechaza sin sustituir el caso.

Cuadro 7.2: Requisitos no funcionales, referidos al modelo de calidad ISO/IEC 25010.

7.4 Casos de uso

El actor principal es el profesional sanitario. Los casos de uso describen sus objetivos sobre el sistema, tratado como caja negra: no importan las clases internas, sino lo que el profesional puede hacer. El caso de uso principal es revisar la medicación de un paciente con apoyo STOPP/START. De él se desprenden la captura (RF01–RF03, RF15), la consulta de avisos (RF08) y las salidas (RF09–RF11). Dos subcasos ocurren siempre que cambia el caso, sin que el profesional los invoque aparte: **evaluar criterios** (RF04) y **persistir el caso** en el navegador. La exportación JSON sí es una acción explícita. La figura K.4 del apéndice K resume estas relaciones.

7.5 Análisis

El análisis de dominio distingue el **caso clínico** que evalúa el motor del **catálogo de criterios**, que no forma parte del estado de sesión. Para interpretar el alcance se separan tres niveles de información:

- **Disponible en la interfaz:** medicaciones, diagnósticos, los nueve parámetros de analítica y constantes, y marcadores de pestañas revisadas.
- **Disponible solo mediante el JSON del caso:** la ficha PatientInfo (incluidos edad

y sexo) y los campos heredados de dosis y duración de Med; la interfaz no permite capturarlos.

- **Sujeta a revisión manual:** condiciones expresadas solo en el texto del aviso, como determinados umbrales de dosis o duración. El motor puede mostrar el aviso, pero no comprobar automáticamente esa condición con la captura ordinaria.

Esta clasificación no demuestra cuántos de los 190 enunciados oficiales son evaluables en cada nivel. Un *Crit* se relaciona con el caso por evaluación: dado un *PatientCase*, resulta aplicable o no. La figura 7.1 conserva únicamente entidades y cardinalidades; los atributos y la secuencia se detallan en el capítulo 8.

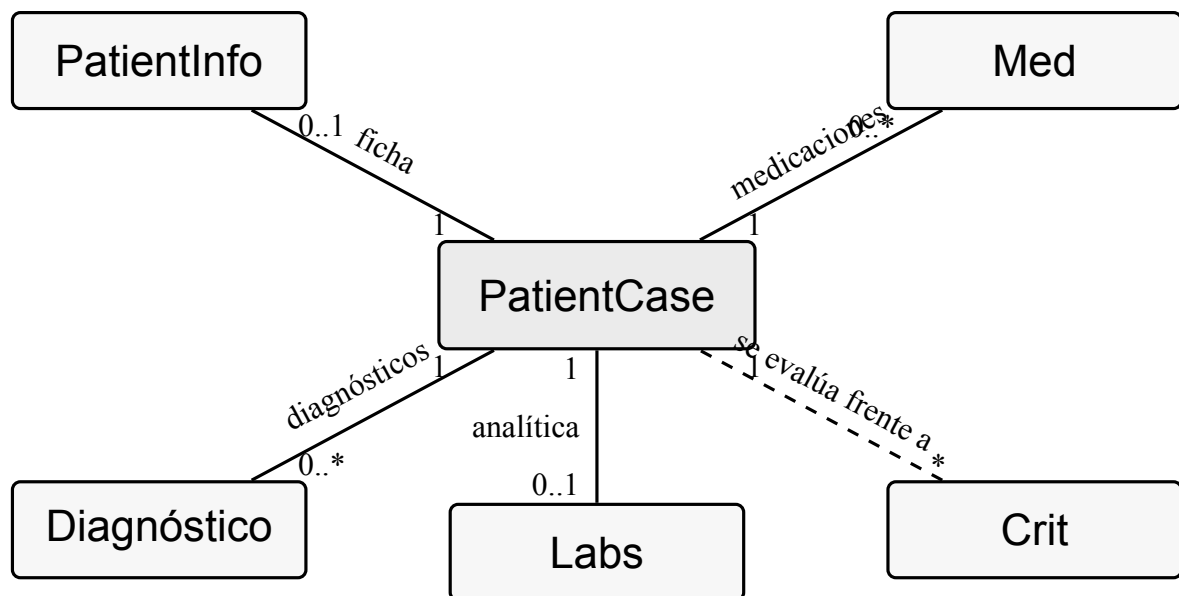


Figura 7.1: Entidades del dominio, con el nombre de cada relación sobre la línea y las multiplicidades en los extremos. *Crit* pertenece al catálogo; el resto constituye el caso de sesión.

Capítulo 8

Diseño del software

Este capítulo concreta la arquitectura del capítulo 5 en dos vistas. La estática fija los tipos del dominio y la forma de un criterio; la dinámica muestra cómo un cambio en el caso se propaga hasta la interfaz y hasta el PDF. Las decisiones de presentación —y su evolución con el tutor— cierran el capítulo. El esquema de fichero y la persistencia con implicaciones de privacidad se tratan en el capítulo 9.

8.1 Diseño estático

8.1.1 Modelo de dominio

El agregado evaluable es `PatientCase`. Agrupa la ficha opcional del paciente (`PatientInfo`: nombre, edad, sexo y campos demográficos auxiliares), la lista de diagnósticos como códigos normalizados, las medicaciones activas y la analítica. Un `Med` lleva identificador, clases farmacológicas y, en el modelo, dosis y duración; esos umbrales no se editan en la interfaz —viven en el texto del criterio—, pero el tipo los conserva para casos JSON antiguos. `Labs` reúne solo las constantes que algún criterio lee (filtración, iones, presión, frecuencia, QTc, TSH). El caso también anota qué pestañas de medicación y de diagnóstico se marcaron como revisadas.

`Crit` no forma parte del agregado persistente. Es el tipo del aviso: identidad, clase STOPP o START, sistema, resumen y, cuando aplica, lógica, relevancia y exclusiones. Los resultados no se guardan en el estado de sesión; se recalculan. La figura 8.1 muestra estas relaciones. El intercambio entre sesiones queda en el JSON versionado, descrito en el capítulo 9.

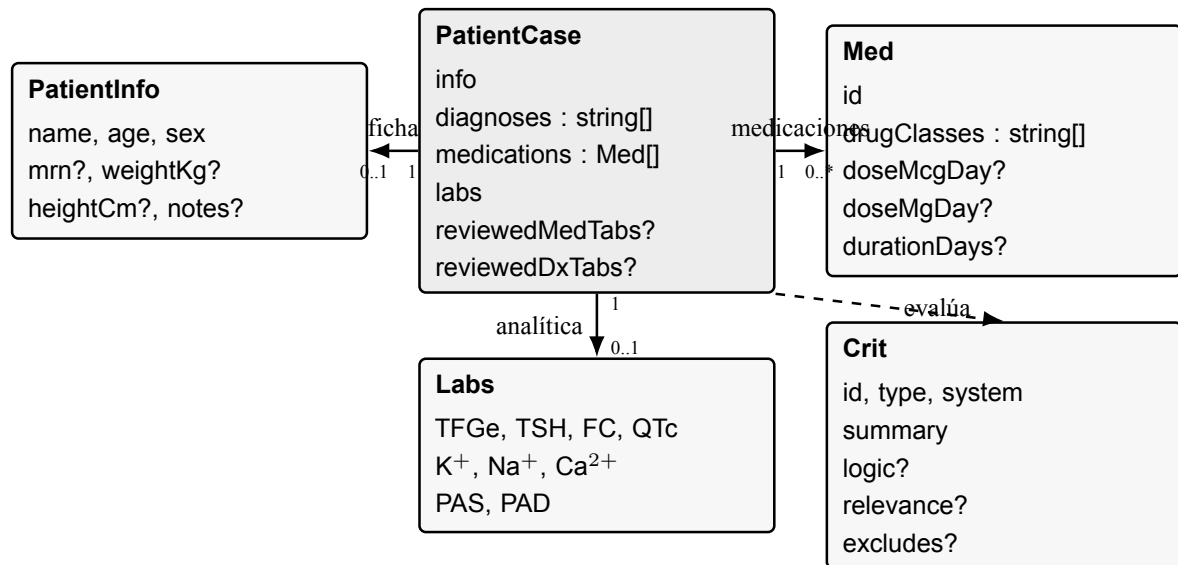


Figura 8.1: Modelo de dominio simplificado. Crit no se persiste con el caso: el motor lo produce al evaluar.

CaseStoreService materializa el agregado como señales y expone patientCase como proyección. CaseIoService consume esa proyección directamente; los pasos la mutan campo a campo y entregan a ReportService los datos necesarios para el informe. PatientInfo existe en el modelo para el informe y el JSON, pero no hay pantalla de captura demográfica.

8.1.2 Estructura de un criterio

Un criterio, en tanto tipo, tiene siete piezas. La tabla 8.1 las enumera. No se describe aquí el esquema del fichero —eso es materia del capítulo 9— sino qué significa cada campo para el diseño.

Campo	Papel
id	Identidad estable (p. ej. un subcriterio STOPP-B).
type	STOPP (prescripción inapropiada) o START (omisión).
system	Sistema fisiológico; alinea el aviso con las pestañas.
summary	Texto clínico del aviso, incluidos umbrales de dosis o duración.
logic	Árbol JSON Logic que el motor aplica al caso.
excludes	Fármacos o clases cuya adición activaría una advertencia de prescripción potencialmente inapropiada. El nombre se conserva por ser parte del contrato técnico.
relevance	Clases que la UI debe destacar cuando el índice automático no basta.

Cuadro 8.1: Campos de Crit a nivel de tipo.

JSON Logic se eligió porque la regla es un valor serializable: el mismo árbol viaja en el catálogo, entra en la prueba y se evalúa sin compilar una función TypeScript por criterio [9]. El motor registra operadores de dominio en un solo punto; el tipo `Crit` no necesita conocerlos.

8.2 Diseño dinámico

8.2.1 Ciclo de evaluación

El flujo crítico es el que sigue a marcar o desmarcar un fármaco. La figura I.1 lo descompone. El clínico pulsa una casilla; `MedsStepComponent.toggleDrug` escribe en `store.meds` una lista nueva (añade el Med del catálogo o lo retira). Esa escritura invalida el `computed applicableCriteria`, que llama a `evaluate` con la instantánea del caso. El motor normaliza diagnósticos y medicaciones y devuelve los `Crit` que se cumplen. La plantilla reparte el resultado en paneles STOPP y START.

El motor también expone `getExcludedMedications`: para cada criterio con `excludes`, simula añadir el fármaco candidato y comprueba si la regla dispararía. Esta capacidad está cubierta por pruebas, pero `MedsStepComponent` no la invoca y la plantilla no muestra ese resultado. El índice `relevance`, ya construido en el arranque, alimenta el cubo «relevantes de otros sistemas» y, si el fármaco marcado era foráneo, el resaltado de grupos y avisos relacionados. El diagrama de esa secuencia está en el apéndice I (figura I.1).

El paso de diagnósticos replica el mismo esquema (`toggleDiagnosis` → `diagnoses` → `evaluate`), con dos matices: las variantes de una familia (p. ej. hipertensión) son mutuamente excluyentes, y un diagnóstico puede deshabilitarse si falta la medicación que lo habilita.

8.2.2 Variante: exportar PDF

La figura I.2 resume la exportación. El clínico pide el informe; el paso vuelve a evaluar el caso —no reutiliza un caché— y entrega paciente, diagnósticos, medicaciones y `Crit[]` a `ReportService`. Este compone el documento con `pdfmake` [12] y el navegador descarga el fichero. Un error de generación se comunica con un *snackbar*; no se escribe nada en el estado de sesión. El diagrama correspondiente es la figura I.2 del apéndice I.

8.2.3 Ciclo de persistencia

El servicio de sesión rehidrata cada señal en el constructor —lectura de `localStorage`, o valor vacío si la clave no existe o el parseo falla— y registra un *effect* que serializa el valor en cada cambio. Si la escritura falla (cuota, modo incógnito), el error se silencia y la sesión sigue en memoria. Las claves de versiones anteriores que ya no se usan se eliminan al arrancar. El tratamiento de esos datos como información de salud se razona en el capítulo 9.

8.3 Diseño de la interfaz

La interfaz no es un listado de 218 avisos: es un espacio de captura en el que los criterios aparecen al lado de lo que el profesional está marcando. Cinco decisiones de presentación sostienen ese encaje. Las decisiones de plataforma (SPA, JSON, señales) no se reiteran; están en el capítulo 5.

Dos cubos por pestaña. Cada pestaña muestra primero los grupos que le son propios y, debajo, un cubo de ítems de otros sistemas que algún criterio de la pestaña activa hace relevantes. Así un AINE puede aflorar en cardiovascular sin obligar a saltar de pestaña, y el profesional ve por qué está ahí. La relevancia transversal (analgésicos, caídas, anticolinérgicos) no genera casillas foráneas: solo dispara avisos.

Pestañas verticales. Con once a trece categorías, una barra horizontal exigía desplazamiento lateral y ocultaba el sistema activo. El raíl vertical —orientación por defecto, reversible en opciones— cabe entero, reserva ancho según la etiqueta más larga de cada paso y escala con la fuente.

Escala de fuente. Tres factores (1, 1,15 y 1,3) se aplican como variable CSS `--font-scale` y se persisten. El clínico agranda el texto sin que el raíl parta etiquetas ni el panel de avisos pierda jerarquía.

Analítica fija en «Otros». No hay paso de laboratorio. El panel vive en la pestaña «Otros» de diagnósticos y muestra siempre todos los campos que algún criterio puede leer. Varios START se disparan por la *ausencia* de un fármaco ante una constante alterada: si el campo solo apareciera «cuando toca», ese aviso sería inalcanzable.

Umbrales en el summary. Dosis y duración (AAS por encima de 100 mg/día, benzodiacepina más de cuatro semanas) no se piden en la casilla. El criterio salta al marcar el fármaco —y el diagnóstico, si aplica— y el umbral queda escrito en el aviso para revisión manual del profesional. Por tanto, el sistema no verifica automáticamente que se supere dicho umbral; la interfaz prioriza una captura breve, pero no se ha medido su efecto en el tiempo de consulta ni en la usabilidad.

8.3.1 Iteraciones de diseño con el tutor

La interfaz actual no nació en un solo trazo. Las rondas con el tutor informático fueron correcciones de encaje, no una lista de tickets.

En mayo se cerró el layout de dos columnas —captura a la izquierda, avisos a la derecha— y el flujo en dos pasos. Un asistente más largo habría separado la pregunta de la respuesta; el tutor insistió en que el criterio debía verse en el mismo gesto de marcar. Esa decisión es la que todavía organiza la pantalla.

En junio la revisión de UI afinó el estado de cada pestaña: badges de conteo, el conmutador «revisada» cuando no hay selección, y la distinción visual entre panel vacío, panel con avisos y pestaña ya recorrida. Sin esos estados, el profesional no sabía si una categoría estaba limpia o simplemente no la había abierto.

En julio las pestañas pasaron a raíl vertical. La barra superior no escalaba: al crecer el catálogo, el sistema activo desaparecía del viewport. El raíl lateral resolvió el problema de un golpe y dejó la barra horizontal como preferencia, no como omisión.

Más adelante se añadieron los *snackbars* de relación y el resaltado bidireccional entre cubos. Marcar un fármaco foráneo —o uno propio emparejado con otro sistema— debía enseñar *qué* falta y *dónde* está, no solo encender un aviso. El snackbar nombra el criterio y el hueco; el pulso visual señala el grupo compañero. Ese refuerzo nace de usar la herramienta en voz alta con el tutor: el enlace clínico existía en el motor y no se veía.

Cada cambio redujo un roce concreto —respuesta lejos de la pregunta, pestaña muda, sistema fuera de pantalla, relación invisible— y dejó el flujo de dos pasos intacto.

Capítulo 9

Gestión de datos e información

Este capítulo describe qué datos maneja la aplicación, cómo se representa el catálogo de criterios, dónde persiste el caso de sesión y cómo se componen los informes. El algoritmo de evaluación y los operadores del motor se tratan en los capítulos 5 y 8; aquí solo importa la información que entra, se guarda y sale.

9.1 Datos manejados por el sistema

El objeto evaluable es el `PatientCase`. Agrupa cuatro bloques clínicos y metadatos de progreso de revisión.

Paciente. El tipo `PatientInfo` contempla nombre, edad, sexo (F/M), número de historia, peso, talla y notas. Ese modelo viaja en la exportación JSON y puede rehidratarse al importar un caso. La interfaz, sin embargo, no incluye un paso de ficha demográfica: la captura cotidiana se limita a medicación, diagnósticos y analítica. Los campos de `PatientInfo` quedan, en uso normal, a `null`; un JSON antiguo o elaborado a mano puede traerlos y el sistema los conserva.

Medicación. Cada `Med` tiene un identificador (nombre del fármaco en el catálogo, o un código `otro__` seguido del identificador de grupo para el ítem personalizado) y una lista de clases farmacológicas. Los campos opcionales de dosis y duración existen solo por compatibilidad con casos exportados; la interfaz no los edita y ningún criterio vigente los lee. Los umbrales de dosis o duración, cuando la guía los menciona, viven en el texto del aviso para que el profesional juzgue si aplican.

Diagnósticos. Se almacenan como códigos normalizados (`string[]`), no como texto libre de la etiqueta visible. Las variantes de una familia —p. ej. las cuatro de hipertensión— son códigos distintos y mutuamente excluyentes en la interfaz.

Analítica y constantes. `Labs` contiene únicamente los parámetros que algún criterio lee. La glucosa, el perfil lipídico, la creatinina aislada y el INR se retiraron porque no alimentaban ninguna regla ni se pedían en el panel. La tabla 9.1 lista el conjunto actual, coincidente

con `lab-capture.ts`. Todos los campos son numéricos anulables: un valor ausente no se inventa.

Campo	Etiqueta en la UI	Uso típico
<code>egfr_ml_min_173</code>	TFGe (ml/min/1,73 m ²)	Umbrales renales STOPP/START
<code>tsh_uUl</code>	TSH (μU/ml)	Criterios tiroideos
<code>fc_lpm</code>	Frecuencia cardíaca (lpm)	Bradicardia / frecuencia
<code>qtc_ms</code>	Intervalo QTc (ms)	STOPP-B15 y afines
<code>potasio_mmol_l</code>	Potasio (mmol/l)	Hiperpotasemia (p. ej. B12)
<code>sodio_mmol_l</code>	Sodio (mmol/l)	Electrolitos (tiazidas)
<code>calcio_corregido_mmol_l</code>	Calcio corregido (mmol/l)	STOPP renales / metabólicos
<code>pas_mmhg</code>	Presión arterial sistólica (mmHg)	START-B1, STOPP-B14
<code>pad_mmhg</code>	Presión arterial diastólica (mmHg)	START-B1

Cuadro 9.1: Parámetros de Labs capturados en la pestaña «Otros» de diagnósticos.

El caso incluye además `reviewedMedTabs` y `reviewedDxTabs`: identificadores de pestañas marcadas como revisadas sin selección (sección 7.2, RF07).

La existencia de un campo en el modelo no implica que esté disponible en la captura ordinaria. Medicaciones, diagnósticos y los nueve valores de Labs se introducen en la UI; `PatientInfo` y los campos heredados de dosis y duración solo pueden llegar mediante un JSON; los umbrales que figuran únicamente en `summary` requieren revisión manual. En consecuencia, una regla dependiente de edad o sexo puede ser evaluable con un JSON que aporte esos datos y, a la vez, ser inaccesible desde la interfaz. Tampoco se considera automática una comprobación de dosis o duración que el profesional deba confirmar leyendo el aviso.

9.2 Modelo de datos de los criterios

Las reglas clínicas no están compiladas en TypeScript: residen en `src/assets/data/criteria.json`, un objeto cuya propiedad `criteria` contiene el array de objetos `Crit`. Cada entrada tiene los campos de la tabla J.1 (apéndice J).

La guía oficial STOPP/START v3 enumera 190 criterios (133 STOPP y 57 START) [2], [3]. El fichero implementado contiene **218 entradas ejecutables** (166 STOPP y 52 START). La diferencia no es un segundo listado clínico: varios ítems oficiales se desglosan en sub-criterios cuando la guía agrupa fármacos o situaciones que, en reglas, deben evaluarse por separado. Ejemplos: STOPP-B12 se parte en variantes IECA y ARA-II; STOPP-B13, en tres combinaciones de ahorradores de potasio; STOPP-B14, en hipotensión frente a nitratos;

STOPP-B19, en AINE frente a corticoide. El identificador conserva el código de sección de la guía, de modo que el profesional reconoce B12 o B14 en la columna de avisos.

El recuento de entradas no acredita por sí solo correspondencia completa con los 190 enunciados: un enunciado puede estar desglosado, requerir datos no capturables en la UI o precisar confirmación manual. Este capítulo describe el catálogo almacenado, no aporta una matriz de cobertura clínica.

Los trece valores de `system` coinciden con los de la guía: indicación de la medicación, cardiovascular, anticoagulantes/antiagregantes, sistema nervioso central, renal, gastrointestinal, respiratorio, musculoesquelético, urogenital, endocrino, riesgo de caídas, analgésicos y carga antimuscarínica/anticolinérgica.

El listado J.1 del apéndice J muestra una entrada real. STOPP-B1 se activa si el paciente toma un fármaco de clase DIGOXINA y tiene el diagnóstico `ic_funcion_sistolica_conservada`; `excludes` indica qué marcaría el motor como medicación potencialmente inapropiada en ese contexto. No equivale a una prohibición clínica absoluta.

9.3 Persistencia

No hay base de datos ni API de pacientes. El estado de la sesión se serializa en `localStorage` del origen de la aplicación, porque el sistema es una SPA sin backend (capítulo 5). Cada señal de dominio tiene su clave; un efecto escribe el JSON en cada cambio y, si el valor es nulo, borra la clave. Al arrancar, el servicio de sesión rehidrata las señales con `JSON.parse`; un error de lectura deja el valor por defecto y la aplicación sigue utilizable. Los fallos de escritura (cuota, modo incógnito) se silencian: el caso permanece en memoria durante la sesión.

Clave	Contenido	Notas
<code>patient</code>	<code>PatientInfo</code> o ausente	Ficha; habitualmente vacía.
<code>diagnoses</code>	Array de códigos	Siempre array al rehidratar.
<code>meds</code>	Array de <code>Med</code>	Siempre array al rehidratar.
<code>labs</code>	Objeto <code>Labs</code> o ausente	Panel de analítica.
<code>reviewedMedTabs</code>	Array de ids	En memoria es un <code>Set</code> .
<code>reviewedDxTabs</code>	Array de ids	Igual que la anterior.
<code>activeSystemTab</code>	Cadena	Pestaña activa; no es dato clínico.
<code>font-scale</code>	1 / 1.15 / 1.3	Preferencia de visualización.
<code>tabs-orientation</code>	<code>vertical</code> / <code>horizontal</code>	Preferencia de visualización.

Cuadro 9.2: Claves de `localStorage` vigentes.

Al iniciar se eliminan claves de versiones anteriores que ya no se usan (`results`,

activeSystem). collapsedSections es estado efímero de interfaz y no se persiste.

La rehidratación no valida el esquema Zod: ese contrato se aplica al *importar* un fichero. En `localStorage`, un JSON que no pueda parsearse cae al valor por defecto, pero un valor sintácticamente válido con estructura inesperada puede entrar en memoria. Por ello el almacenamiento del navegador no se considera una frontera de confianza ni aporta garantía de integridad.

9.4 Privacidad y protección de datos

Medicación, diagnósticos y analítica son datos de salud y, por tanto, categoría especial en el sentido del art. 9 del Reglamento (UE) 2016/679 [15], [16]. El diseño del TFG no crea un fichero de pacientes en un servidor ni transmite el caso a un encargado de tratamiento: no hay cuenta de usuario, no hay API y la evaluación no sale del navegador. El PDF y el JSON se generan en el cliente y solo abandonan el equipo si el profesional los guarda o los envía por sus propios medios.

Ese planteamiento **evita el tratamiento en un servidor de aplicación**, pero no garantiza por sí solo confidencialidad, integridad ni borrado. El profesional sigue siendo quien introduce el caso y decide si exporta una copia.

Confidencialidad y copias. Quien acceda al perfil del navegador —por uso compartido del equipo, hurto, sincronización o copia de seguridad del perfil— puede leer `localStorage`, que la aplicación no cifra. PDF y JSON también se generan sin cifrado: al descargarlos quedan fuera del control de la aplicación y pueden permanecer en descargas, copias de seguridad, correo u otros destinos elegidos por el profesional. El texto copiado queda sujeto al comportamiento del portapapeles y del sistema operativo.

Integridad y corrupción. Un actor con acceso al perfil o a las herramientas del navegador puede manipular las claves locales. El JSON ilegible se sustituye por el valor por defecto, pero la rehidratación no aplica Zod y no detecta necesariamente un objeto parseable con tipos o códigos alterados. Esto puede cambiar el caso evaluado o provocar un comportamiento no previsto. En cambio, los ficheros importados sí se validan antes de llamar a `loadCase`; esa validación estructural no acredita que los datos clínicos sean verdaderos.

Retención y borrado. No existe caducidad automática: cerrar el navegador no elimina el caso. La retención dura hasta confirmar «reiniciar», borrar los datos del sitio o retirar el perfil. El reinicio sustituye el contenido clínico restaurable por un caso vacío; los efectos de persistencia pueden conservar claves técnicas con arrays o valores vacíos. La aplicación no puede verificar el borrado físico interno del navegador ni retirar copias sincronizadas, respaldos

o ficheros ya exportados. Estos últimos deben gestionarse y eliminarse en sus respectivos destinos.

Uso previsto y riesgo residual. Al abrir la guía rápida no se adjuntan datos del caso: el enlace al PDF de Sacyl [3] es público y no lleva parámetros clínicos. Aun así, por la persistencia sin cifrado, la falta de validación de integridad al rehidratar y la ausencia de control sobre exportaciones y copias, esta implementación de TFG no debe emplearse con datos reales identificables sin una evaluación organizativa y técnica adicional. El riesgo residual incluye acceso local no autorizado, manipulación silenciosa, retención superior a la prevista y divulgación de una exportación; la ausencia de backend reduce la superficie de exposición, pero no elimina esos riesgos.

9.5 Generación de informes

Hay tres salidas independientes, disparadas por el profesional. La secuencia de construcción se detalla en el capítulo 8; aquí se describe el contenido.

PDF. El informe incluye: cabecera con logotipo (si el recurso está disponible) y marca «STOPP/START — Optimización terapéutica»; título «Informe STOPP/START» y fecha; dos columnas con diagnósticos (etiquetas) y medicaciones; y la sección «Criterios aplicables», con una tabla STOPP y otra START. Cada tabla tiene columnas ID, sistema y descripción (summary). El pie recuerda que se trata de una herramienta de apoyo y que el criterio clínico es imprescindible. El nombre de fichero sigue `stopp-start_<nombre>.pdf` (o paciente si no hay nombre).

JSON versionado. La exportación construye un objeto `CaseExport` con `version` igual a la literal `1.0`, `exportedAt` en ISO y el `patientCase` completo. La importación parsea el fichero y lo valida con un esquema Zod: versión distinta de `1.0`, tipos incorrectos o Labs vacío como objeto sin las nueve claves se rechazan y el caso en curso no se sustituye. Las claves de analítica retiradas en casos antiguos se descartan y el resto carga. El fichero se nombra `stopp-start_<nombre>_<fecha>.json`.

Texto plano. La copia al portapapeles agrupa los avisos por sistema, con líneas STOPP B1 - - - (o START) seguidas del summary. No incluye ficha ni medicación: es un recordatorio de criterios, no un informe completo.

Capítulo 10

Pruebas

10.1 Estrategia de pruebas

La batería de pruebas comprueba el comportamiento técnico de las reglas frente a casos derivados de la interpretación de la guía: una regla debe activarse en el escenario positivo descrito y permanecer inactiva en el negativo. El método de desarrollo dirigido por pruebas ya se justificó en el capítulo 3; este capítulo describe *qué* se comprueba y qué evidencia aporta, sin equipararla a una validación clínica.

Se distinguen cuatro niveles. Las *pruebas unitarias* cubren funciones puras (agrupación, visibilidad de grupos, procedencia de ítems foráneos), taxonomías de medicamentos y diagnósticos, el almacén del caso y el motor genérico (operadores, normalización, exclusión preventiva). Las de *integración* ejercitan el catálogo real `criteria.json`, el esquema Zod de importación y las guardas de integridad entre reglas y catálogo. Las de *interfaz* cubren los dos pasos de captura (`MedsStep` y `DiagnosisStep`): selección, cubos propios y foráneos, navegación y acciones de salida. La *verificación de conformidad con la guía*, tratada en sección propia, reúne casos en los que cada especificación describe un caso (medicación, diagnósticos y, cuando aplica, constantes) y afirma si el criterio debe disparar.

El runner es Karma [17] con Jasmine [18] sobre TypeScript, en Chrome sin interfaz gráfica. No hay servidor ni base de datos: todo el comportamiento relevante es reproducible en el navegador de pruebas.

10.2 Pruebas unitarias

El motor genérico (`criteria-engine.service.spec.ts`, 22 casos) comprueba `evaluate` con reglas sintéticas: coincidencia de diagnósticos sin distinguir mayúsculas, filtros que dejan fuera lo que no aplica, umbrales de analítica, tolerancia a lógica ausente o mal formada, y los operadores `inDrugClass` y `multiple*`. También cubre `getExcludedMedications`: el motor devuelve un fármaco candidato si, al simularlo sobre el caso, el criterio dispararía. Esta capacidad no está integrada en la interfaz actual.

Los helpers puros evitan duplicar esa lógica en los componentes. Se verifica que `groupBySystem` conserva el orden de primera aparición, que `isMedGroupChecked` / `isDxGroupChecked` reconocen tanto el catálogo como la opción «Otro», y que el cálculo

de cubos (*group-visibility*) y de enlaces entre pestañas (*foreign-provenance*) respeta relevancia específica frente a la transversal. Las taxonomías y el índice de relevancia se prueban como datos: sistemas, clases con miembros, familias diagnósticas (HTA) y dependencias que habilitan un diagnóstico solo cuando hay medicación predisponente. El almacén (*CaseStoreService*) cubre reset, pestañas revisadas y persistencia en *localStorage* sin mutar el caso a espaldas de las señales.

10.3 Pruebas de integración

Tres frentes enlazan piezas que, aisladas, no garantizarían coherencia.

En primer lugar, las especificaciones clínicas no inventan reglas: importan el *criteria.json* de producción (*ALL_CRITERIA*, *crit(id)*) y llaman a *evaluate* sobre ese catálogo. Un cambio en un árbol *json-logic* rompe el caso correspondiente sin necesidad de un doble de la regla.

En segundo lugar, la importación de un caso JSON v1.0 pasa por un esquema Zod (*caseExportSchema*). Las 24 pruebas de *case-io* rechazan JSON mal formado, versiones no soportadas y cargas parciales que dejarían el almacén inconsistente, y comprueban que el mensaje de error no expone el volcado técnico de Zod. El informe PDF y el texto plano al portapapeles se cubren en el mismo bloque de entrada/salida.

En tercer lugar, *criteria-data-integrity.spec.ts* es una guarda de suite: todo identificador de *excludes.medications* existe en el catálogo de fármacos; toda clase usada en lógica o exclusiones tiene al menos un miembro; todo código de *DIAGNOSIS_MAP* está referenciado o figura en una lista blanca comentada; las clases decorativas (sin criterio) también están listadas de forma explícita; y la política de variantes de HTA (las cuatro en criterios generales; solo *hta/hta_no_complicada* en B5) queda fijada por test. El script *scripts/audit-criteria.cjs* complementa esas guardas fuera de Karma: recorre criterios, medicamentos y diagnósticos en el sistema de ficheros y emite inconsistencias (clases o códigos desconocidos, referencias por nombre en lugar de por clase).

10.4 Pruebas de interfaz

Los pasos de medicamentos y diagnósticos se prueban montando el componente, no simulando su plantilla a mano. En medicamentos (20 casos) se comprueba la selección compartida de un fármaco multiclase entre su pestaña principal y otra relevante, que un unitario que afloja por relevancia no se cuenta además en «Otros», la ausencia de panel de dosis/duración, la navegación por teclado, los cubos y el enlace B19 (diurético de asa ↔ AINE), y que un fallo al copiar o al exportar PDF muestra un aviso. En diagnósticos (32 casos) se cubre la familia mutuamente excluyente de HTA (una sola variante activa), el conteo de «Otro» sin

fila fantasma, el panel fijo de analítica siempre visible en la pestaña «Otros» —independiente de cualquier selección—, la escritura y el borrado de una constante sin pisar las demás, y la misma dualidad de orientación de pestañas (raíl vertical por defecto, barra superior si se elige). El cascarón (rutas, diálogo de reinicio, guía rápida, tooltip) añade 12 casos de contrato de navegación y accesibilidad.

10.5 Verificación de conformidad con la guía STOPP/START

Cada caso expresa una interpretación de la guía STOPP/START v3 [2], [3]; comprueba la conformidad entre esa interpretación y la regla implementada, pero no su validez clínica. El patrón es uniforme. Se construye un `PatientCase` con fábricas compartidas (`aine()`, `digoxina()`, `makeLabs()`, etc.), se localiza el criterio real por identificador estable (`crit('STOPP-B20-...')`) y se afirma `evaluate(...).length` igual a uno o a cero. Donde la guía exime un fármaco, el caso negativo es tan importante como el positivo: D12 no debe disparar con quetiapina o clozapina solas; C16 no debe disparar si hay enfermedad cardiovascular establecida.

Hay un fichero por sección fisiológica (`criteria-a.spec.ts...criteria-m.spec.ts`), 544 casos en total. La densidad no es uniforme —B, C, D, E y K concentran más escenarios; F, G y M son más delgadas—, pero las trece secciones STOPP y los START asociados tienen especificación propia. Esta distribución no demuestra por sí sola una correspondencia completa con los 190 criterios oficiales. Sí cierra el hueco que una revisión intermedia señaló: I, J, K y M carecían de `spec` y agrupaban varios de los defectos A2, A6, A9 y A14 descritos más adelante.

El valor de esta batería se midió al corregir el motor: el caso que reproducía el defecto se escribió primero; la regla en `criteria.json` se ajustó después; la misma prueba impide reintroducir el falso positivo o el falso negativo.

10.6 Inventario estático y estado de ejecución

El recuento de la tabla 10.1 es un inventario estático de las cláusulas `it(` presentes en `src/**/*.spec.ts`: 985 en total. No es el resultado de una ejecución. No se ha registrado una ejecución final de Karma para el estado documentado. La documentación operativa del repositorio cita una ejecución anterior de alrededor de 807 especificaciones, previa a varias ampliaciones, pero ese antecedente no permite atribuir un resultado a las 985 cláusulas actuales. El inventario no encontró `xit/fit`. Por módulo, el desglose estático es: criterios A–M, 544; motor genérico, 22; catálogo y taxonomías (incluida integridad), 195; ayudas de agrupa-

ción y visibilidad, 98; almacén y preferencias, 28; entrada/salida (JSON, PDF, portapapeles), 34; pasos de captura, 52; cascarón de interfaz, 12.

Tipo de prueba	N.º	Estado
Conformidad con la guía (criterios A–M)	544	No ejecutada actualmente
Unitarias (motor, helpers, catálogo, almacén)	338	No ejecutada actualmente
Integración (integridad, Zod, informes)	39	No ejecutada actualmente
Interfaz (pasos y cascarón)	64	No ejecutada actualmente
Total inventariado	985	Resultado no disponible

Cuadro 10.1: Inventario estático de la batería (conteo de cláusulas `it( en los *.spec.ts)`, sin resultado de ejecución actual. El script `audit-criteria.cjs` no forma parte de este total.

10.7 Amenazas a la validez

La guía fue interpretada principalmente por el autor al traducir sus enunciados a datos, reglas y casos de prueba. Una misma interpretación incorrecta puede quedar reflejada tanto en la regla como en su test y producir un resultado coherente entre ambos, pero no conforme con la guía. No se dispone de una revisión clínica independiente completa de la correspondencia entre los 190 criterios oficiales y las 218 entradas del catálogo.

Tampoco se han realizado pruebas con pacientes ni una evaluación formal con profesionales. Por ello, esta batería no permite concluir utilidad clínica, impacto asistencial, comprensión de los avisos ni usabilidad. Además, al no haber una ejecución actual de la suite, el inventario estático no demuestra que las pruebas compilen o terminen satisfactoriamente en el estado documentado.

10.8 Errores detectados y corregidos

Durante el desarrollo, la batería permitió detectar reglas que producían un aviso distinto del esperado en los casos modelados. Dos campañas lo ilustran.

La auditoría de STOPP cardiovascular contra la guía eliminó nueve subcriterios que no existen en STOPP/START v3 o que duplicaban un criterio oficial. Cinco eran inventados (STOPP-B2-DRONEDARONA-IC-NYHA, STOPP-B2-ITRACONAZOL-IC, STOPP-B4-AMIODARONA-PROLONGA-QTC, STOPP-B6-DRONEDARONA-FA-PERMANENTE, STOPP-B15-AMIODARONA-HTA-QTC). Cuatro eran redundantes o estaban fuera del texto oficial (antiarrítmico de primera línea en FA; tres variantes de hiperpotasemia que atribuían a B12/B13 lo que la guía reserva a IECA/ARA-II o a una combinación). Cada eliminación

se reflejó en `criteria-b.spec.ts`: el identificador dejó de resolver y el caso asociado desapareció.

Una revisión posterior del motor (hallazgos A3–A14) corrigió lógica que sí correspondía a un criterio oficial pero estaba mal formalizada. El apéndice G detalla cinco casos por su efecto sobre el aviso (B20, C16, D12, J3 y C12): cada uno tiene en el código un caso positivo y otro negativo, cuyo resultado actual queda pendiente de ejecutar. Otros ajustes del mismo lote —B6 acotado a amiodarona (clase III), I7 restringido a duloxetina en lugar de toda la clase ISRN, unificación de códigos de caídas en la sección K— siguen el mismo patrón: el test nombra el fármaco o el diagnóstico que la guía distingue y falla si la regla vuelve a generalizar. Estos casos expresan resultados funcionales observables, no solo líneas cubiertas; su conformidad final queda sujeta a las amenazas a la validez descritas.

Capítulo 11

Manual de usuario

11.1 Requisitos mínimos

La aplicación es una página web que se ejecuta por completo en el navegador. No requiere cuenta ni instalación de cliente una vez servidos los ficheros estáticos. No se ha aportado una URL de despliegue público; por ello, este manual solo documenta la puesta en marcha local verificable desde el repositorio. El profesional necesita:

- Un navegador compatible con la compilación de Angular 20 y con JavaScript activado. No se documenta una matriz de navegadores ensayada, por lo que no se afirma compatibilidad certificada con versiones concretas.
- Una pantalla que permita manejar el formulario y el panel de resultados. El código incluye adaptaciones de disposición, pero no se aporta una campaña de pruebas por resolución; la respuesta visual debe comprobarse en el entorno de uso antes de emplear la herramienta.
- Conectividad solo para cargar la página y el catálogo de criterios. El caso no se envía a ningún servidor.

Para ejecutar o modificar el proyecto en local se requiere, además, una versión de Node.js compatible con Angular 20 (^20.19.0, ^22.12.0 o ^24.0.0) y npm. No hace falta base de datos ni ningún servicio en segundo plano.

11.2 Puesta en marcha local para evaluación

Desde la raíz del repositorio:

```
npm install
npx ng serve --host 127.0.0.1 --port 4200
```

Estas órdenes levantan un **servidor de desarrollo**; no constituyen una instalación de producción ni acreditan un despliegue final. El servidor queda en `http://127.0.0.1:4200`. El navegador abre la pestaña de medicamentos. La configuración de desarrollo es la determinada de `ng serve`; no se utiliza la compilación de producción para trabajar.

Otras órdenes útiles:

- Compilación de desarrollo: `npx ng build --configuration development`.

- Batería de pruebas, una sola pasada, Chrome sin ventana: `npx ng test --watch=false --browsers=ChromeHeadless`.

La persistencia del caso en curso vive en el `localStorage` del origen (127.0.0.1:4200 en local). Borrar los datos del sitio equivale a reiniciar el caso.

11.3 Guía de uso

El flujo tiene dos pasos y un panel de resultados visible junto a la captura en la disposición descrita por la interfaz. No hay pantalla de datos demográficos: se seleccionan medicamentos, se completan diagnósticos y, si procede, las constantes de la pestaña «Otros»; los avisos STOPP y START se actualizan al vuelo. Las figuras de esta sección son esquemas de interfaz (*wireframes*), no capturas. En el momento del cierre documental no se aportaron capturas reales ni un registro de ejecución manual; las conductas descritas se contrastan con el código y las especificaciones automáticas localizadas en el repositorio, cuya ejecución final debe registrarse por separado.

El recorrido habitual es: (1) marcar los fármacos activos; (2) leer los avisos que ya puedan dispararse solo con medicación; (3) pasar a diagnósticos; (4) rellenar la analítica en «Otros» cuando un criterio dependa de TFG_e, presión arterial, iones u otras constantes; (5) exportar PDF, copiar el texto, guardar el JSON o reiniciar.

11.3.1 Caso ficticio reproducible

El siguiente caso no representa a una persona real ni es una recomendación terapéutica. Su finalidad es permitir que otra persona reproduzca un recorrido mínimo y compare el resultado con una regla concreta del catálogo:

1. Pulsar *Reiniciar* y confirmar *Limpiar* para partir de un caso vacío.
2. Mantener vacía la selección de medicamentos.
3. En *Diagnósticos*, abrir el sistema cardiovascular y el grupo *Hipertensión*; marcar *HTA (sin especificar)*.
4. Comprobar que el panel muestra el criterio START-B1-ANTIHIPERTENSIVO-HTA, que propone considerar un antihipertensivo cuando hay HTA y no consta uno en el caso.
5. Volver a *Medicamentos*, seleccionar *Amlodipino* y comprobar que ese criterio deja de resultar aplicable.

El resultado de los pasos 3–5 se deduce de `criteria.json` y de los casos positivo y negativo de `criteria-b.spec.ts`. No se presenta como una ejecución manual fechada. Para conservar evidencia de una ejecución real deberán anotarse fecha, versión, navegador y resultado observado, y adjuntarse las capturas o exportaciones producidas.

Salidas esperadas antes de seleccionar amlodipino. *Copiar criterios* debe producir texto que incluya START, el sistema cardiovascular, el código B1 y el resumen del criterio. El PDF debe titularse «Informe STOPP/START» e incluir el diagnóstico y la tabla START. *Guardar* debe descargar un JSON versión 1.0 cuya parte relevante tenga esta forma (la fecha de exportación y los campos de revisión dependen de la sesión):

```
{
  "version": "1.0",
  "exportedAt": "<fecha ISO de la ejecución>",
  "patientCase": {
    "info": null,
    "diagnoses": ["hta"],
    "medications": [],
    "labs": null
  }
}
```

Estos ejemplos describen formato y contenido esperado según `case-io.service.spec.ts`, `clipboard-text.spec.ts` y `report.service.spec.ts`; no sustituyen los ficheros generados en una ejecución documentada.

11.3.2 Pantalla de medicamentos

La figura H.1 resume el primer paso. La cabecera alterna entre Medicamentos y Diagnósticos y ofrece *Visualización* (orientación de las pestañas de sistema) y *Exportar PDF*. La barra de herramientas permite guardar el caso en JSON, cargar uno previo, abrir la guía rápida y reiniciar —esta última acción pide confirmación—.

A la izquierda, un raíl de sistemas (Cardiovascular, Neurológico, Renal, etc., más «Otros») filtra el catálogo. Dentro de cada sistema hay dos bloques: los grupos propios de esa pestaña y, debajo, los *relevantes de otros sistemas*, que la aplicación muestra porque algún criterio de la pestaña activa los cita. Marcar un fármaco en el bloque foráneo no duplica la selección: el mismo identificador se comparte entre pestañas. Si el principio activo no está listado, la opción *Otro* del grupo añade un ítem con la clase farmacológica del grupo, de modo que los criterios que miran esa clase puedan disparar.

A la derecha, el panel *Criterios activados* se actualiza al marcar o desmarcar. Un contador en la barra resume medicaciones y diagnósticos ya elegidos. Cada pestaña puede marcarse como revisada cuando no hay selección, para dejar constancia de que se miró y no aplica nada. El esquema de esta pantalla está en el apéndice H (figura H.1).

11.3.3 Pantalla de diagnósticos y analítica

La figura H.2 muestra el segundo paso, con la misma cabecera y el mismo panel de criterios. Las pestañas corresponden a sistemas diagnósticos (Cardiovascular, Neurológico, ...). Dentro de cada una vuelven a aparecer el cubo propio y el de diagnósticos relevantes de otros sistemas. Algunas familias —la hipertensión— se presentan como opciones excluyentes: marcar «HTA grave» desmarca «HTA moderada».

No hay un paso aparte de datos del paciente. La analítica y las constantes (TFGe, presión arterial sistólica y diastólica, frecuencia, QTc, potasio, sodio, calcio corregido, TSH) viven en un panel fijo de la pestaña *Otros*. El panel no depende de lo que esté marcado: un START como el de antihipertensivo ante PAS/PAD elevadas se evalúa por la ausencia de fármaco, y ninguna casilla «activaría» esos campos. Si un valor no se rellena, los criterios que lo leen no disparan. El esquema está en la figura H.2 del apéndice H.

11.3.4 Resultados y generación del informe

El panel derecho (figura H.3) es el resultado: no hay una tercera pantalla. Los avisos START (omisión de tratamiento indicado) van encima; los STOPP (prescripción potencialmente inapropiada), debajo. Cada bloque muestra un contador y agrupa las tarjetas por sistema orgánico, plegables. El texto de cada tarjeta es el summary del criterio, con umbrales de dosis o duración cuando la guía los menciona —el profesional juzga si aplican; la interfaz no pide esos datos.

Desde cualquier paso se puede:

- **Exportar PDF:** informe del caso y de los criterios activados, generado en el navegador.
- **Copiar criterios:** texto plano al portapapeles, útil para pegarlo en la historia clínica.
- **Guardar / Cargar:** fichero JSON versión 1.0 (medicación, diagnósticos, analítica). Un JSON ajeno o de versión distinta se rechaza con un mensaje comprensible.
- **Reiniciar:** vacía el caso de `localStorage` tras confirmar.

El esquema del panel está en la figura H.3 del apéndice H.

11.4 Resolución de errores

La tabla F.1 del apéndice F resume síntomas frecuentes de puesta en marcha, exportación y evaluación, con la comprobación y la acción recomendadas. No sustituye el diagnóstico clínico cuando un criterio esperado no aparece: en ese caso hay que contrastar con la guía y registrar el caso para revisión.

11.5 Preguntas frecuentes

¿Esta herramienta sustituye al juicio clínico? No. Es un cribado: señala criterios STOPP/START que *podrían* aplicar al caso introducido. La decisión de deprescribir, iniciar o mantener un tratamiento es siempre del profesional, a la luz de la situación concreta y de la guía [2], [3].

¿Dónde se guardan los datos? En el `localStorage` del navegador, en el origen desde el que se sirve la página. No hay servidor de aplicación ni cuenta de usuario. El contenido no sale del equipo salvo que se exporte un JSON o un PDF.

¿Puedo cargar un caso ya revisado? Sí. *Guardar* descarga un JSON v1.0; *Cargar* lo restaura (medicación, diagnósticos y analítica). El fichero debe ser una exportación de esta aplicación; un documento arbitrario se rechaza.

¿Y si el fármaco o el diagnóstico no está en el catálogo? En el grupo correspondiente se marca *Otro*. El ítem hereda la clase del grupo (medicamentos) o un código asociado al grupo (diagnósticos), de forma que los criterios que razonan por clase o por familia puedan evaluarse. No inventa un principio activo nuevo en el catálogo maestro.

Capítulo 12

Principales aportaciones

Este capítulo pone en correspondencia cada objetivo específico del capítulo 2 con un resultado comprobable del sistema. No se reclama una validación clínica experimental: la comprobación se limita al inventario de pruebas automáticas y a la verificación de conformidad con la guía oficial [2], [3], tal como delimita el OE7.

12.1 Cumplimiento de los objetivos

Objetivo	Aportación concreta
OE1	Catálogo declarativo en <code>criteria.json</code> : las situaciones STOPP/START v3 modeladas se expresan como reglas, no como ramas embebidas en el código del motor. Las 218 entradas no acreditan por sí solas la cobertura de los 190 criterios oficiales.
OE2	Motor de inferencia sobre <code>json-logic-js</code> con operadores de dominio; evalúa los avisos con medicación, diagnósticos, analítica y los datos importados consultados por cada regla. En el catálogo actual, solo la edad se lee entre los datos demográficos; dosis y duración requieren confirmación manual.
OE3	Flujo guiado en dos pestañas (medicaciones y diagnósticos), con captura auxiliar de analítica en la pestaña «Otros», sobre un catálogo acotado. No incluye ficha demográfica ni captura de dosis o duración.
OE4	Relevancia automática derivada del catálogo (grupos propios frente a relevantes de otros sistemas) y deshabilitado de diagnósticos que no pueden activar ningún criterio con la selección actual. El motor incluye un cálculo auxiliar basado en <code>excludes</code> , pero no se expone en la interfaz.
OE5	Presentación de los criterios activados, agrupados por sistema, en las propias pestañas de trabajo, con el enunciado del aviso listo para la revisión.
OE6	Informe PDF, exportación e importación del caso en JSON versionado, copia en texto plano y persistencia local en el navegador, sin almacenamiento en servidor.
OE7	Inventario de pruebas automáticas (TDD) y casos de conformidad con la guía v3, reforzado con guardas de integridad del catálogo. La suite actual no se ha ejecutado en el cierre documental; no constituye ensayo clínico ni revisión independiente con perfil asistencial.

Cuadro 12.1: Correspondencia entre los objetivos específicos (capítulo 2) y las aportaciones del trabajo.

12.2 Aportaciones técnicas

La decisión de mantener las reglas fuera del código —detallada en el capítulo 5— permite auditar, corregir o ampliar un criterio editando el catálogo, sin reescribir el evaluador. Sobre esa base, la relevancia por pestaña y las advertencias sobre medicamentos se calculan a partir de las propias reglas, de modo que la interfaz no exige un mapa manual paralelo que se desalinearía con la guía.

El segundo pilar técnico es tratar los tests como especificación ejecutable de la interpretación realizada [7]: la batería contiene casos que afirman si determinados avisos deben activarse o no. Las guardas de catálogo (consistencia entre criterios, fármacos y diagnósticos)

detectan referencias huérfanas. El inventario, su estado de ejecución y los hallazgos de esa batería se recogen en el capítulo 10.

12.3 Aportaciones al ámbito clínico

La aportación al ámbito clínico es un prototipo que transforma en avisos las reglas STOPP/START evaluables con los datos disponibles. La interfaz aporta medicación, diagnósticos y determinados valores analíticos; un JSON importado puede aportar edad y sexo; y los umbrales de dosis o duración permanecen en el texto para confirmación manual. Las reglas que requieren otros datos no son evaluables por esas vías. La herramienta no prescribe ni sustituye el juicio clínico; muestra posibles prescripciones potencialmente inapropiadas y omisiones de tratamientos indicados.

El alcance permanece el declarado en el capítulo 2: catálogo acotado, sin historia clínica electrónica y sin ensayo sobre pacientes reales. Lo que se aporta es un apoyo a la decisión reproducible, no una evidencia de impacto asistencial, utilidad o usabilidad.

Capítulo 13

Conclusiones

13.1 Conclusiones técnicas

Tres decisiones de diseño facilitaron el desarrollo documentado. La primera es mantener las reglas de decisión fuera del código: el motor aplica `criteria.json` en lugar de codificar STOPP/START v3 como ramas de programa. Esa separación, descrita en el capítulo 5, hizo posible auditar el cardiovascular contra la guía [2], [3] y eliminar criterios inventados o redundantes sin reescribir el evaluador.

La segunda es usar los tests como especificación ejecutable de la interpretación realizada, no solo como red de seguridad del software [7]. Formular un caso y su resultado esperado antes de implementar la regla convirtió esa expectativa en un contrato comprobable. El capítulo de pruebas separa el inventario estático de 985 cláusulas de su resultado: la suite actual no se ha podido ejecutar en el entorno de cierre y, por tanto, no se presenta como superada. También recoge las amenazas derivadas de compartir interpretación entre regla y prueba y de no disponer de revisión independiente completa.

La tercera es la arquitectura de página única sin servidor. En el contexto de un TFG que maneja datos de salud, evaluar y persistir el caso solo en el navegador evita un backend de pacientes y reduce la exposición asociada a ese componente, aunque no elimina los riesgos del almacenamiento local. El coste, aceptado en el capítulo 2, es no cubrir escenarios multiusuario ni despliegue institucional.

Los límites técnicos son igual de concretos. El catálogo de fármacos y diagnósticos está acotado a lo modelado: no cubre el arsenal terapéutico completo. El modelo de datos admite edad y sexo, pero la interfaz no ofrece un paso de ficha demográfica: esos campos solo llegan por importación JSON. La analítica sí se captura en la pestaña «Otros» del paso de diagnósticos. Así, la evaluación automática desde la UI se limita a medicación, diagnósticos y esos valores; la importación JSON añade los datos demográficos, aunque el catálogo actual solo consulta la edad; las condiciones de dosis y duración exigen confirmación manual; y los criterios dependientes de otros datos quedan sin evaluar. El recuento del catálogo (218 subcriterios en `criteria.json`) no coincide 1:1 con los 190 enunciados de la guía: algunos se desglosan en varias reglas y no existe una matriz que demuestre la correspondencia completa. Por último, las dependencias diagnósticas y las familias de variantes se modelaron con mayor detalle en cardiovascular e hipertensión; el resto de sistemas no tiene el mismo grado

de especialización.

13.2 Conclusiones personales

El trabajo se desarrolló con dos perfiles de tutoría distintos: el tutor del área informática y la cotutora clínica. Esa dualidad obligó a traducir de forma continua entre restricciones de software (alcance, tests, interfaz) y matices de la guía. Cuando una duda clínica no bloqueaba el desarrollo, se documentó y se siguió con la parte técnica; cuando una petición de interfaz amenazaba el calendario, se dejó en cuarentena en lugar de fusionarla. Un ejemplo de ese filtro es el selector jerárquico de variantes, del que solo se incorporó la familia de hipertensión.

Haría dos cosas de otro modo. La primera: registrar las horas desde el primer día. La dedicación se ha reconstruido a posteriori a partir del histórico de Git y de las rondas de trabajo, y esa reconstrucción es menos precisa que un registro contemporáneo. La segunda: anteponer el paso de datos del paciente (edad, sexo y constantes) a refinamientos de la taxonomía. Varios criterios del motor ya dependían de esos campos mientras la UI aún no permitía introducirlos; corregir ese desajuste antes habría alineado el modelo, las pruebas y el flujo de entrada.

En conjunto, el proyecto muestra la importancia de acotar el alcance, hacer comprobables las reglas y declarar de forma explícita lo que la evidencia disponible todavía no permite sostener. Sin pacientes, revisión clínica independiente completa ni evaluación formal de usabilidad, no se concluye utilidad clínica ni mejora del trabajo en consulta.

Capítulo 14

Vías de trabajo futuro

Las líneas siguientes convierten el fuera de alcance del capítulo 2 en trabajo posterior concreto: qué falta, por qué importa y qué haría falta.

14.1 Paso de datos del paciente en la interfaz

Falta un paso de captura de edad y sexo. El modelo ya los admite y la analítica se introduce en «Otros», pero sin ficha demográfica los criterios que dependen de esos datos (p. ej. STOPP-B16) no se activan partiendo únicamente de la UI; actualmente solo pueden recibirlos mediante importación JSON. Haría falta un *patient-info-step* como primer paso de la ruta y pruebas de los umbrales de edad de la guía.

14.2 Generalización de variantes y dependencias diagnósticas

Falta extender a todos los sistemas el tratamiento de variantes y de dependencias diagnóstico–medicación, hoy modelado con mayor detalle en cardiovascular e hipertensión. Sin eso, otras pestañas muestran opciones que no pueden disparar ningún criterio. Haría falta generalizar el mecanismo y, después, revisión con perfil clínico para completar cada sistema.

14.3 Integración con la historia clínica electrónica

Falta la conexión con la HCE: el profesional introduce a mano fármacos y diagnósticos. Preseleccionar fármacos y diagnósticos desde la historia clínica es el escenario excluido en la sección 2.3; su efecto sobre el tiempo de uso tendría que medirse. Haría falta un convenio asistencial, un adaptador de identificadores y una política de minimización compatible con el diseño sin servidor.

14.4 Normalización con SNOMED CT y entrada en lenguaje natural

Falta abandonar el catálogo cerrado como única vía de entrada. Un texto libre debería resolverse contra SNOMED CT [19] y, desde ahí, contra las reglas, con el objetivo de reducir errores de selección de la etiqueta interna. Haría falta un mapa SNOMED ↔ catálogo, búsqueda, y validación humana antes de evaluar si se aspira al lenguaje natural.

14.5 Revisión profesional, dudas clínicas y catálogo

Faltan tres cierres que el OE7 dejó fuera (capítulos 2 y 10). Una evaluación con profesionales —sesiones de casos, no una validación clínica ni un ensayo— para medir comprensión, usabilidad y desacuerdos sobre los avisos; harían falta un protocolo, métricas definidas y, si hay datos reales, un marco ético. Resolver con la cotutora las dudas ya documentadas: fragilidad como indicador de esperanza de vida en STOPP-B16, antiarrítmicos de clase Ia ausentes del catálogo, y unificación de las variantes de insuficiencia cardíaca. Ampliar los fármacos poco usados en España pero citados en la guía, con clases y tests, sin crear clases huérfanas.

14.6 Matriz de correspondencia con la guía

Falta demostrar la relación entre los 190 criterios oficiales y las 218 entradas del catálogo. Haría falta revisar cada criterio con perfil clínico y registrar sus reglas asociadas, datos necesarios, casos positivo y negativo y estado: evaluable desde la interfaz, evaluable mediante importación JSON, sujeto a confirmación manual, no evaluable o no implementado. Esta matriz permitiría delimitar la cobertura sin inferirla a partir del número de reglas o de pruebas.

Referencias

- [1] R. L. Maher, J. Hanlon y E. R. Hajjar, “Clinical consequences of polypharmacy in elderly”, *Expert Opinion on Drug Safety*, vol. 13, nº 1, págs. 57-65, 2014. DOI: 10.1517/14740338.2013.827660.
- [2] D. O’Mahony, A. Cherubini, A. R. Guiteras et al., “STOPP/START criteria for potentially inappropriate prescribing in older people: version 3”, *European Geriatric Medicine*, vol. 14, nº 4, págs. 625-632, 2023. DOI: 10.1007/s41999-023-00777-y.
- [3] Gerencia Regional de Salud de Castilla y León. “Criterios STOPP/START versión 3 (español)”. (2023), dirección: https://www.saludcastillayleon.es/profesionales/es/calidad-seguridad-paciente/seguridad-paciente/medicacion-dano/adherencia-tratamiento-revision-plan-terapeutico/criterios-stopp-start.ficheros/3057685-criterios-stopp-start_v3_espa%C3%B1ol.pdf (visitado 11-08-2026).
- [4] L. Monteiro, T. Maricoto, I. Solha, I. Ribeiro-Vaz, C. Martins y M. Monteiro-Soares, “Reducing Potentially Inappropriate Prescriptions for Older Patients Using Computerized Decision Support Tools: Systematic Review”, *Journal of Medical Internet Research*, vol. 21, nº 11, e15385, 2019. DOI: 10.2196/15385.
- [5] D. O’Mahony, A. Gudmundsson, R. L. Soiza, M. Petrovic, A. J. Cruz-Jentoft, A. Cherubini et al., “Prevention of Adverse Drug Reactions in Hospitalized Older Patients with Multi-morbidity and Polypharmacy: The SENATOR Randomized Controlled Clinical Trial”, *Age and Ageing*, vol. 49, nº 4, págs. 605-614, 2020. DOI: 10.1093/ageing/afaa072.
- [6] M. R. Blum, B. T. G. M. Sallevelt, A. Spinewine, D. O’Mahony, E. Moutzouri, M. Feller et al., “Optimizing Therapy to Prevent Avoidable Hospital Admissions in Multimorbid Older Adults (OPERAM): Cluster Randomised Controlled Trial”, *The BMJ*, vol. 374, n1585, 2021. DOI: 10.1136/bmj.n1585.
- [7] K. Beck, *Test-Driven Development: By Example*. Boston: Addison-Wesley, 2002.
- [8] Angular Team. “Angular”. (2026), dirección: <https://angular.dev> (visitado 16-08-2026).
- [9] J. Wadhams. “JSON Logic”. (2015), dirección: <https://jsonlogic.com/> (visitado 16-08-2026).
- [10] ReactiveX. “RxJS: Reactive Extensions Library for JavaScript”. (2024), dirección: <https://rxjs.dev/> (visitado 16-08-2026).

- [11] Angular. “Angular Material”. (2026), dirección: <https://material.angular.dev> (visitado 16-08-2026).
- [12] B. Pampuch. “pdfmake”. (2024), dirección: <https://pdfmake.github.io/docs/> (visitado 16-08-2026).
- [13] C. McDonnell. “Zod”. (2025), dirección: <https://zod.dev> (visitado 16-08-2026).
- [14] ISO/IEC, *ISO/IEC 25010:2023. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*, International Organization for Standardization, Geneva, 2023.
- [15] Parlamento Europeo y Consejo de la Unión Europea. “Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos y por el que se deroga la Directiva 95/46/CE (RGPD)”. (2016), dirección: <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32016R0679> (visitado 16-08-2026).
- [16] Agencia Española de Protección de Datos. “Guía para profesionales del sector sanitario”. (2022), dirección: <https://www.aepd.es/recurso-multimedia/guia-para-profesionales-del-sector-sanitario> (visitado 16-08-2026).
- [17] Karma. “Karma: Spectacular Test Runner for JavaScript”. (2023), dirección: <https://karma-runner.github.io/> (visitado 16-08-2026).
- [18] Pivotal Labs. “Jasmine: Behavior-Driven JavaScript”. (2025), dirección: <https://jasmine.github.io/> (visitado 16-08-2026).
- [19] SNOMED International. “SNOMED CT Starter Guide”. (), dirección: <https://docs.snomed.org/snomed-ct-practical-guides/snomed-ct-starter-guide> (visitado 16-08-2026).

Apéndice A

Comandos de operación

Este anexo resume los comandos necesarios para instalar, ejecutar, comprobar y auditar el prototipo desde la raíz del repositorio. El recorrido de uso se describe en el capítulo 11; aquí solo se documenta la operación técnica. El servidor de desarrollo debe lanzarse con la configuración `development`; no se emplea la compilación de producción como entorno de trabajo.

Comando	Propósito
<code>npm install</code>	Instala las dependencias declaradas en <code>package.json</code> .
<code>npx ng serve --host 127.0.0.1 --port 4200</code>	Arranca el servidor de desarrollo (configuración <code>development</code>) en el puerto 4200.
<code>npx ng test --watch=false --browsers=ChromeHeadless</code>	Ejecuta la batería Karma + Jasmine en Chrome sin interfaz, una sola pasada.
<code>npx ng build --configuration development</code>	Genera el artefacto de desarrollo (comprobación de tipos incluida).
<code>bash scripts/check-links.sh</code>	Verifica el patrón de documentación enlazada (@linked): enlaces rotos, documentos huérfanos y ficheros fuera del mapa.
<code>node scripts/audit-criteria.cjs</code>	Audita la consistencia entre <code>criteria.json</code> , el catálogo de fármacos y el de diagnósticos (clases y códigos huérfanos).

Cuadro A.1: Comandos de instalación, ejecución, prueba y auditoría.

El script `scripts/check-links.sh` existe en el repositorio y debe terminar con código de salida 0. Las líneas de consola del tipo «Error evaluando criterio» que puedan aparecer durante `ng test` corresponden a una especificación deliberada de manejo de errores, no a un fallo de la suite: el veredicto es la línea final de totales.

Apéndice B

Sistemas clínicos y recuento de criterios

La guía STOPP/START v3 formula 190 enunciados (133 STOPP y 57 START) [2]. El catálogo de la aplicación no los reproduce 1:1: algunos enunciados se desglosan en varias reglas (subcriterios) y otros se agrupan. La tabla B.1 resume, por los 13 sistemas del fichero `criteria.json`, cuántas reglas STOPP y START hay implementadas. No se listan los textos completos: el volumen duplicaría la memoria sin añadir información de diseño.

Los recuentos corresponden a la versión del catálogo presente en el corte documental del 16 de agosto de 2026 (218 reglas: 166 STOPP y 52 START). Son cifras de implementación, no un censo de la guía impresa [3].

Sistema clínico	STOPP	START	Total
Indicación de la medicación	8	0	8
Sistema cardiovascular	35	11	46
Anticoagulantes/Antiagregantes	19	2	21
Sistema nervioso central	31	7	38
Sistema renal	10	4	14
Sistema gastrointestinal	9	7	16
Sistema respiratorio	4	3	7
Sistema musculoesquelético	11	9	20
Sistema urogenital	8	5	13
Sistema endocrino	11	1	12
Riesgo de caídas	14	0	14
Analgésicos	5	3	8
Carga antimuscarínica/anticolinérgica	1	0	1
Total	166	52	218

Cuadro B.1: Reglas implementadas por sistema clínico (catálogo `criteria.json`).

Apéndice C

Trazabilidad entre requisitos y evidencia

La tabla C.1 enlaza cada requisito con evidencia localizada en el repositorio. «Localizada» significa que existe una especificación automática que expresa ese comportamiento; no significa que la suite se haya ejecutado con éxito para esta edición de la memoria. «Parcial» identifica requisitos cuya dimensión completa exige medición, auditoría o prueba de uso adicional. Salvo las rutas que comienzan por `src/`, los nombres de fichero se refieren a `src/app/`.

Cuadro C.1: Matriz requisitos–evidencia del repositorio.

ID	Comportamiento trazado	Evidencia localizada	Estado documental
RF01	Seleccionar y deseleccionar medicamentos	<code>steps/meds-step/meds-step.component.spec.ts</code> ; <code>core/group-checked.spec.ts</code>	Localizada; ejecución final no registrada
RF02	Seleccionar diagnósticos y variantes excluyentes	<code>steps/diagnosis-step/diagnosis-step.component.spec.ts</code> ; <code>core/data/diagnosis-variants.spec.ts</code>	Localizada; ejecución final no registrada
RF03	Capturar analítica y constantes	<code>core/lab-capture.spec.ts</code> ; <code>steps/diagnosis-step/diagnosis-step.component.spec.ts</code>	Localizada; ejecución final no registrada
RF04	Reevaluar tras cambios del caso	<code>core/services/criteria-engine.service.spec.ts</code> ; especificaciones de ambos pasos	Localizada; ejecución final no registrada
RF05	Calcular advertencias preventivas de medicación	<code>core/services/criteria-engine.service.spec.ts</code>	Solo motor; no integrado en la interfaz; ejecución final no registrada
RF06	Mostrar elementos relevantes de otros sistemas	<code>core/data/system-relevance.spec.ts</code> ; <code>core/group-visibility.spec.ts</code>	Localizada; ejecución final no registrada
RF07	Marcar pestañas revisadas y persistir el estado	<code>core/case-store.service.spec.ts</code>	Localizada; ejecución final no registrada
RF08	Agrupar resultados por tipo y sistema	<code>core/criteria-groups.spec.ts</code> ; especificaciones de ambos pasos	Localizada; ejecución final no registrada
RF09	Generar y descargar el informe PDF	<code>core/report.service.spec.ts</code>	Localizada; no se adjunta PDF ejecutado
RF10	Copiar criterios como texto plano	<code>core/clipboard-text.spec.ts</code> ; especificaciones de ambos pasos	Localizada; ejecución final no registrada
RF11	Exportar e importar JSON validado	<code>core/case-io.service.spec.ts</code>	Localizada; no se adjunta JSON ejecutado
RF12	Reiniciar solo tras confirmación	<code>confirm-reset-dialog.component.spec.ts</code> ; <code>core/case-store.service.spec.ts</code>	Localizada; ejecución final no registrada
RF13	Escala de fuente y orientación persistentes	<code>core/display-settings.service.spec.ts</code> ; especificaciones de ambos pasos	Localizada; ejecución final no registrada
RF14	Guía rápida y enlace a la guía oficial	<code>quick-guide-dialog.component.spec.ts</code>	Localizada; ejecución final no registrada
RF15	Opción «Otro» por grupo	<code>core/group-checked.spec.ts</code> ; especificaciones de ambos pasos	Localizada; ejecución final no registrada
RNF01	Flujo en dos pasos con resultados junto a la captura	<code>app.routes.spec.ts</code> ; especificaciones de ambos pasos	Parcial: estructura localizada; usabilidad no evaluada

Cuadro C.1: Matriz requisitos–evidencia del repositorio (cont.).

ID	Comportamiento trazado	Evidencia localizada	Estado documental
RNF02	Evaluación en cliente en la misma interacción	core/services/criteria-engine.service.spec.ts especificaciones de ambos pasos	Parcial: sin medición de latencia
RNF03	Tres escalas de fuente	core/display-settings.service.spec.ts	Parcial: sin auditoría de accesibilidad
RNF04	Catálogo JSON separado del motor	src/assets/data/criteria.json; core/data/criteria-data-integrity.spec.ts	Estructura localizada; ejecución final no registrada
RNF05	Persistencia local del caso	core/case-store.service.spec.ts; core/case-io.service.spec.ts	Parcial: persistencia localizada; privacidad no auditada

Apéndice D

Cobertura demostrable del catálogo de criterios

El catálogo contiene 218 reglas JSON, distribuidas como muestra la tabla B.1. Ese recuento se obtiene directamente de `src/assets/data/criteria.json`. Las especificaciones `criteria-a.spec.ts` a `criteria-m.spec.ts` contienen casos por secciones del catálogo, y `criteria-data-integrity.spec.ts` expresa guardas entre reglas y catálogos.

La tabla D.1 es deliberadamente una matriz de *evidencia del repositorio*, no una matriz de equivalencia clínica. La presencia de un fichero por sección tampoco demuestra por sí sola que cada una de sus reglas tenga casos positivo y negativo ejecutados. No se ha validado una correspondencia criterio por criterio entre los 190 enunciados de la guía (133 STOPP y 57 START) y las 218 reglas. Una regla puede desglosar parte de un enunciado, varias reglas pueden agruparse o un criterio puede necesitar datos no capturados. Por tanto, ningún recuento de esta tabla demuestra cobertura completa de la guía.

Cuadro D.1: Matriz de cobertura demostrable del catálogo, sin equivalencia 190→218.

Sistema del catálogo	STOPP	START	Especificación localizada	Correspondencia con guía
Indicación de la medicación	8	0	<code>criteria-a.spec.ts</code>	No validada
Sistema cardiovascular	35	11	<code>criteria-b.spec.ts</code>	No validada
Anticoagulantes/Antiagregantes	19	2	<code>criteria-c.spec.ts</code>	No validada
Sistema nervioso central	31	7	<code>criteria-d.spec.ts</code>	No validada
Sistema renal	10	4	<code>criteria-e.spec.ts</code>	No validada
Sistema gastrointestinal	9	7	<code>criteria-f.spec.ts</code>	No validada
Sistema respiratorio	4	3	<code>criteria-g.spec.ts</code>	No validada
Sistema musculoesquelético	11	9	<code>criteria-h.spec.ts</code>	No validada
Sistema urogenital	8	5	<code>criteria-i.spec.ts</code>	No validada
Sistema endocrino	11	1	<code>criteria-j.spec.ts</code>	No validada
Riesgo de caídas	14	0	<code>criteria-k.spec.ts</code>	No validada
Analgésicos	5	3	<code>criteria-l.spec.ts</code>	No validada
Carga antimuscarínica/anticolinérgica	1	0	<code>criteria-m.spec.ts</code>	No validada
Total del catálogo	166	52	Especificaciones localizadas; ejecución final no registrada	No validada

Para convertir esta matriz en una acreditación de cobertura clínica falta una revisión independiente que enumere los 190 criterios oficiales y, para cada uno, identifique reglas, datos

necesarios, caso positivo, caso negativo y estado (completo, parcial, manual, no evaluable o no implementado). Esa correspondencia no puede inferirse de los totales y no se inventa en esta memoria.

Apéndice E

Criterios de aceptación de los requisitos funcionales

Este anexo complementa el capítulo 7. La tabla E.1 formula cada requisito funcional con identificador, prioridad y descripción completa. La tabla 7.1 del cuerpo solo identifica el enunciado breve. La tabla E.2 agrupa requisitos que comparten procedimiento de comprobación. La comprobación parte de un caso vacío salvo que el propio escenario requiera datos previos. No presupone que la prueba se haya ejecutado ni sustituye la evidencia del capítulo 10.

Cuadro E.1: Requisitos funcionales del sistema implementado.

ID	Prioridad	Descripción
RF01	Alta	El profesional podrá seleccionar y deseleccionar medicamentos del catálogo, organizados por categoría terapéutica (pestañas cardiovascular, anticoagulantes, SNC, renal, gastrointestinal, respiratorio, endocrino, urológico, osteoarticular, antiinfecciosos y «Otros»). Cada fármaco marcado formará parte del caso y podrá desmarcarse en cualquier momento.
RF02	Alta	El profesional podrá seleccionar y deseleccionar diagnósticos del catálogo, organizados por sistema. En las familias con variantes mutuamente excluyentes —en la implementación actual, la hipertensión arterial: HTA sin especificar, no complicada, moderada o grave— marcar una variante retirará las hermanas. Los diagnósticos que dependen de una medicación habilitante permanecerán no seleccionables hasta que dicha medicación esté en el caso.
RF03	Alta	En la pestaña «Otros» del paso de diagnósticos, el profesional podrá introducir, modificar o dejar vacíos los parámetros de analítica y constantes que algún criterio lee: TFGe, TSH, frecuencia cardíaca, QTc, potasio, sodio, calcio corregido, PAS y PAD. El panel será fijo (no condicionado a la selección).

Cuadro E.1: Requisitos funcionales del sistema implementado (cont.).

ID	Prioridad	Descripción
RF04	Alta	Tras cada cambio de medicación, diagnóstico o analítica, el sistema evaluará automáticamente el caso frente al catálogo STOPP/START y actualizará los avisos aplicables en el propio paso, sin petición explícita ni ida y vuelta a un servidor.
RF05	Baja	El motor ofrecerá un cálculo auxiliar de los medicamentos del catálogo que, de añadirse al caso actual, dispararían una regla con campo <i>excludes</i> . Esta capacidad no se presenta en la interfaz actual y queda disponible para una integración posterior.
RF06	Alta	En la pestaña activa, el sistema mostrará un bloque «Relevantes de otros sistemas» con fármacos o diagnósticos de otras pestañas que participan en criterios del sistema en revisión, de modo que el profesional no deba abandonar la pestaña para marcarlos.
RF07	Media	El profesional podrá marcar una pestaña como revisada cuando no tenga selección, para dejar constancia de que la ha examinado y no aplica nada. Si añade una selección, el marcador se retirará. Una pestaña con selección no podrá marcarse como revisada.
RF08	Alta	El sistema mostrará los criterios aplicables agrupados por tipo (STOPP y START) y, dentro de cada tipo, por sistema orgánico, con código de sección y texto del aviso.
RF09	Alta	El profesional podrá descargar un informe PDF del caso con cabecera institucional, fecha, diagnósticos, medicaciones y tablas de criterios STOPP y START aplicables.
RF10	Media	El profesional podrá copiar al portapapeles un texto plano con los criterios aplicables, agrupados por sistema. Si no hay avisos, el texto será «Sin criterios aplicables.».
RF11	Alta	El profesional podrá exportar el caso completo a un fichero JSON versionado e importar un fichero del mismo formato. Un JSON que no supere la validación se rechazará sin alterar el caso en curso.
RF12	Media	El profesional podrá reiniciar el caso. El sistema pedirá confirmación explícita; solo si se confirma se vaciarán medicaciones, diagnósticos, analítica y marcadores de revisión.

Cuadro E.1: Requisitos funcionales del sistema implementado (cont.).

ID	Prioridad	Descripción
RF13	Media	El profesional podrá elegir la escala de fuente (pequeño, mediano o grande) y la orientación de las pestañas (raíl vertical u horizontal). Ambas preferencias se conservarán entre recargas.
RF14	Media	El profesional podrá abrir una guía rápida que explique STOPP y START, el flujo en dos pasos y el enlace al PDF oficial de la guía [3].
RF15	Media	El profesional podrá añadir, en un grupo del catálogo, un ítem «Otro» que no figura como fármaco o diagnóstico nominado, de modo que la clase o el grupo participe en la evaluación aunque el principio activo concreto no esté listado.

Requisitos	Criterio de aceptación observable
RF01–RF03, RF07, RF13 y RF15	Tras realizar la acción descrita desde la interfaz, el valor visible y el caso en memoria reflejan la selección, deselección, captura, revisión o preferencia; al ejecutar la acción inversa se recupera el estado anterior. Las preferencias de RF13 se conservan tras recargar.
RF04 y RF08	Al cambiar una entrada que hace aplicable o deja de hacer aplicable una regla conocida, los paneles STOPP/START se actualizan sin pulsar un botón de cálculo ni enviar el caso a un servidor.
RF05	Una prueba directa del motor devuelve un fármaco ausente cuyo añadido satisfaría una regla con <i>excludes</i> y omite un candidato que no la satisfaría. No se exige una representación en la interfaz.
RF06	Un ítem de otra pestaña que participa en una regla del sistema activo aparece en «Relevantes de otros sistemas» y puede seleccionarse desde ese bloque.
RF09 y RF10	La acción genera, respectivamente, una descarga PDF o texto de portapapeles con los datos y criterios aplicables descritos; el caso sin avisos produce el mensaje especificado en RF10.
RF11	Exportar e importar el fichero generado restaura el caso. Un fichero malformado, de versión distinta o que incumpla el esquema muestra un rechazo y deja intacto el caso previo.
RF12	Cancelar la confirmación conserva el caso; confirmarla vacía sus campos y marcadores persistidos.
RF14	La guía rápida se abre desde la interfaz, contiene las tres materias indicadas y ofrece el enlace al documento oficial.

Cuadro E.2: Criterios de aceptación de los requisitos funcionales.

Apéndice F

Resolución de incidencias de uso

La tabla F.1 complementa el capítulo 11. Recoge síntomas de puesta en marcha, exportación y evaluación, con la comprobación y la acción recomendadas.

Síntoma	Comprobación	Acción
No abre la página local	Revisar la consola de <code>ng serve</code> y que el puerto 4200 no esté ocupado.	Corregir el error mostrado o elegir otro puerto; abrir exactamente la dirección indicada por la CLI.
La interfaz abre sin catálogo	En las herramientas del navegador, comprobar la petición a <code>assets/data/criteria.json</code> .	Servir la aplicación desde su raíz; no abrir directamente <code>index.html</code> como fichero local.
El JSON no se carga	Confirmar que procede de <i>Guardar</i> , que contiene <code>version: "1.0"</code> y un <code>patientCase</code> válido.	No editarlo a mano; volver a exportarlo. Un rechazo no debe reemplazar el caso que ya estaba abierto.
No se copia el texto	El navegador puede denegar el portapapeles por permisos o contexto.	Conceder permiso si la política local lo permite o seleccionar el contenido por otro medio; la interfaz debe mostrar el fallo.
Falla la descarga PDF	Comprobar permisos de descarga y mensajes del navegador.	Reintentar tras habilitar descargas. Como alternativa temporal, usar <i>Copiar criterios</i> o <i>Guardar JSON</i> .
El caso reaparece o desaparece	El estado pertenece al <code>localStorage</code> del origen actual.	Usar siempre el mismo origen para recuperarlo; <i>Reiniciar</i> o borrar los datos del sitio lo elimina. Exportar JSON antes de borrar.
No aparece un criterio esperado	Revisar fármacos, diagnósticos y analítica, y leer los umbrales del aviso. Edad, sexo, dosis y duración no se capturan en la interfaz.	No forzar una conclusión clínica. Contrastar con la guía y registrar el caso para revisión; algunos criterios requieren confirmación manual o datos no disponibles.

Cuadro F.1: Comprobaciones ante incidencias de uso y exportación.

Apéndice G

Detalle de errores detectados en el motor

Este anexo detalla los cinco hallazgos A3–A14 resumidos en el sección 10.8. En todos ellos la lógica correspondía a un criterio oficial pero estaba mal formalizada. El código contiene un caso positivo y otro negativo; su resultado actual queda pendiente de ejecutar.

- **B20 (A3).** La lógica solo exigía antihipertensivo *central*; diuréticos y alfabloqueantes figuraban en el resumen y en las exclusiones, pero no disparaban. Un paciente con estenosis aórtica grave y furosemida no veía el STOPP. La regla es ahora una disyunción de las cuatro clases; los casos esperan activación con furosemida y con doxazosina.
- **C16 (A5).** El AAS en prevención primaria no negaba `cardiopatia_isquemica` ni `ictus_previo`. Un ictus previo con AAS —prevención secundaria correcta— generaba el aviso de prevención primaria, en contradicción con START-C2. Ambas negaciones están en la lógica y en la spec.
- **D12 (A4).** Cualquier neuroléptico en parkinsonismo o Lewy disparaba, incluidas quetiapina y clozapina, que STOPP v3 exime. El paciente bien tratado recibía un STOPP infundado. La lógica combina la clase con la exclusión por identificador; la spec afirma el silencio con quetiapina sola y el disparo si se añade haloperidol.
- **J3 (A6).** Se usaba la clase BETABLOQUEANTE completa en un criterio de no cardioselectivos: bisoprolol disparaba. La lógica actual cita `BETABLOQUEANTE_NO_CARDIOSELECTIVO`; la especificación distingue bisoprolol (no activa la regla) de carvedilol y propranolol.
- **C12 (A1).** El STOPP de ISRS con anticoagulante y antecedentes de sangrado no exigía el ISRS: cualquier anticoagulado con esos antecedentes recibía «evitar ISRS». Se añadió `inDrugClass(ISRS)` y el caso negativo correspondiente.

Apéndice H

Esquemas de interfaz

Los tres esquemas siguientes complementan el capítulo 11. No son capturas de una ejecución fechada: describen la disposición implementada de medicamentos, diagnósticos y el panel de resultados.

STOPP/START	[Medicamentos]	Diagnósticos	Visualización	Exportar PDF
Guardar	Cargar	Guía	Reiniciar	3 medicaciones 1 diagnóstico

Sistemas: Cardiovascular (activa), SNC, Renal ... Otros

Del sistema

IECA Betabloqueantes Diurét. de asa

☐ Enalapril ☐ Ramipril

☒ Bisoprolol ☒ Furosemida

Relevantes de otros sistemas

AINE (Osteo/músculo-esq.)

☐ Ibuprofeno ☐ Naproxeno

☐ Otro

Criterios activados Copiar

START 1

B8 — Considerar iSGLT2

en IC con FE reducida

STOPP 1

B19 — Evitar AINE

en IC con diurético de asa

Figura H.1: Pantalla de medicamentos: pestañas de sistema, lista de fármacos en dos cubos y panel de criterios con acciones PDF/JSON.

STOPP/START	Medicamentos	[Diagnósticos]	Visualización	Exportar PDF
-------------	--------------	----------------	---------------	--------------

Sistemas: ... Urológico, **Otros** (activa)

Analítica y constantes

TFGe (ml/min/1,73 m²) [42] PAS (mmHg) [158]

PAD (mmHg) [92] K⁺ (mmol/l) [4,1]

QTc (ms) [] TSH (μU/ml) []

Diagnósticos (Otros / Geriátrico)

☒ Caídas de repetición

☐ Fragilidad ☐ Otro

Criterios activados

START 1 — B1 antihipertensivo

(PAS > 140 o PAD > 90 y sin tratamiento)

STOPP 1 — K1 benzodiacepina

y caídas de repetición

Figura H.2: Pantalla de diagnósticos: pestaña «Otros» con el panel fijo de analítica y el mismo panel de criterios a la derecha.

Criterios activados

Copiar criterios

START 2

Sistema cardiovascular

B1 — Considerar antihipertensivo (PAS > 140 mmHg / PAD > 90 mmHg)

B8 — Considerar iSGLT2 en insuficiencia cardíaca

STOPP 2

Sistema cardiovascular

B19 — Evitar AINE o corticoide en IC con diurético de asa

Riesgo de caídas

K1 — Evitar benzodiacepina si hay caídas de repetición

Figura H.3: Panel de resultados STOPP/START, agrupado por sistema, visible en ambas pestañas del flujo.

Apéndice I

Diagramas de secuencia

Los diagramas siguientes detallan el capítulo 8: el ciclo al marcar una medicación y la exportación a PDF.

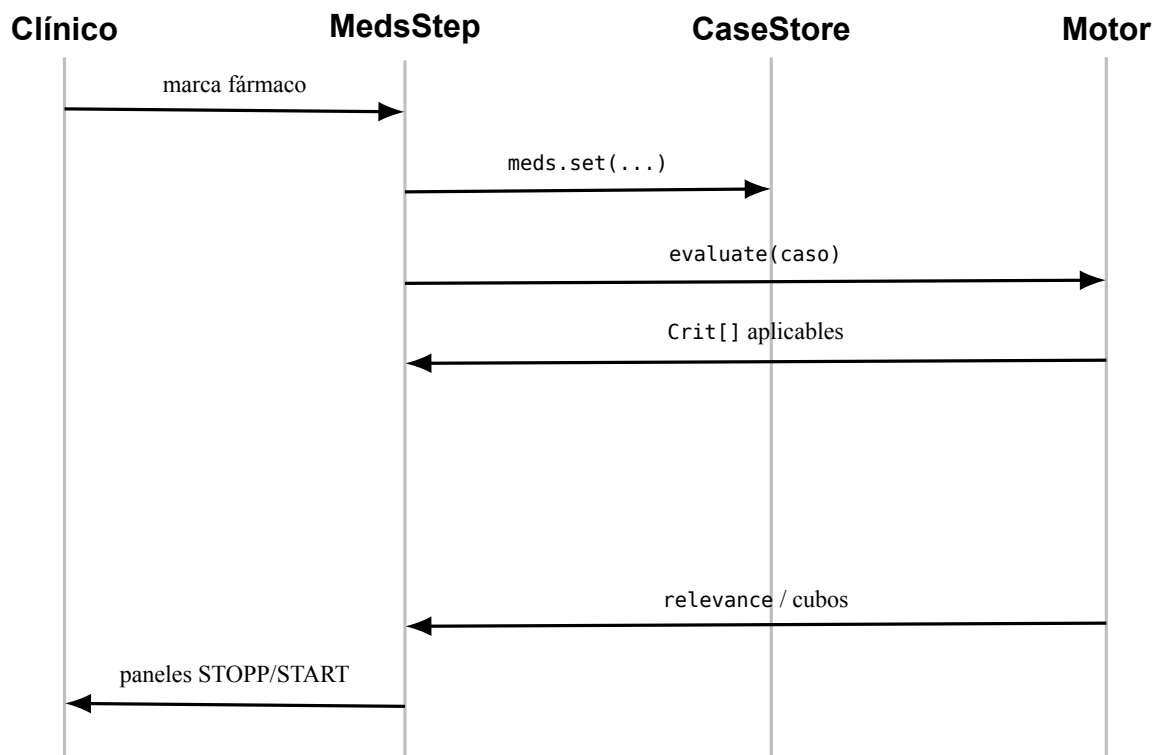


Figura I.1: Secuencia implementada al marcar una medicación: el estado de sesión cambia, el motor evalúa y la UI actualiza criterios y relevancia. El cálculo auxiliar de excludes no está integrado en este flujo.

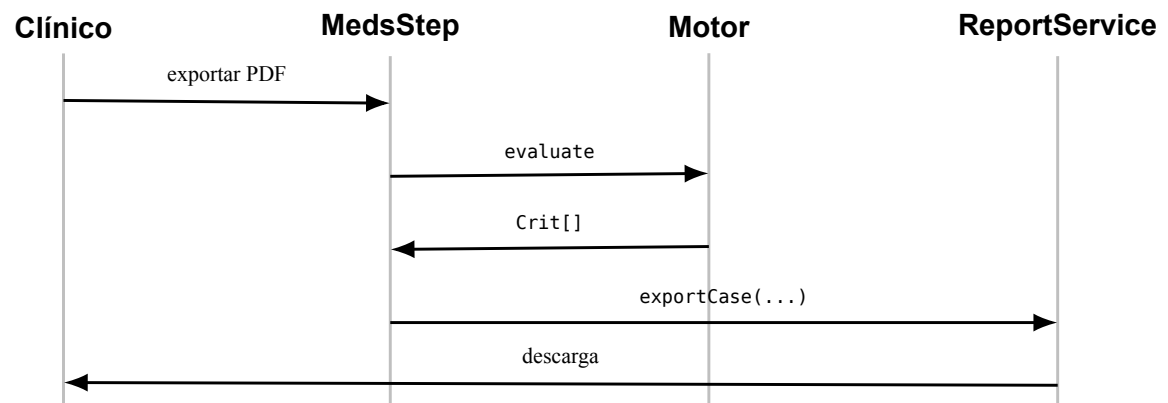


Figura I.2: Secuencia de exportación a PDF: evaluación al vuelo y composición en el cliente.

Apéndice J

Esquema de una regla del catálogo

Este anexo detalla el modelo de sección 9.2: los campos de cada entrada de `criteria.json` y un ejemplo real (STOPP-B1).

Campo	Significado
id	Identificador estable (STOPP-B1-DIGOXINA, ...).
type	STOPP o START.
system	Uno de los trece sistemas del catálogo.
summary	Texto del aviso mostrado al profesional.
logic	Árbol json-logic ejecutable (operadores estándar y personalizados).
excludes	Fármacos o clases cuya adición activaría una advertencia de prescripción potencialmente inapropiada (opcional). Se conserva <code>excludes</code> como nombre técnico del campo.
relevance	Clases extra para el índice de visibilidad cuando no se infieren de la lógica (opcional).

Cuadro J.1: Campos de una entrada de `criteria.json`.

```
{
  "id": "STOPP-B1-DIGOXINA",
  "type": "STOPP",
  "system": "Sistema cardiovascular",
  "summary": "Evitar digoxina en pacientes con insuficiencia
    cardiaca con funcion sistolica ventricular conservada.",
  "logic": {
    "and": [
      {"inDrugClass": ["DIGOXINA", {"var": "medications"}]},
      {"in": ["ic_funcion_sistolica_conservada",
        {"var": "diagnoses"}]}
    ]
  },
  "excludes": {
    "medications": ["Digoxina"],
    "drugClasses": ["DIGOXINA"]
  }
}
```

Listing J.1: Entrada real de `criteria.json`: STOPP-B1.

Apéndice K

Diagramas del cuerpo de la memoria

Los cuatro diagramas siguientes complementan los capítulos 3, 4, 5 y 7. El texto de cada capítulo los interpreta; aquí se agrupan por formato de página completa.

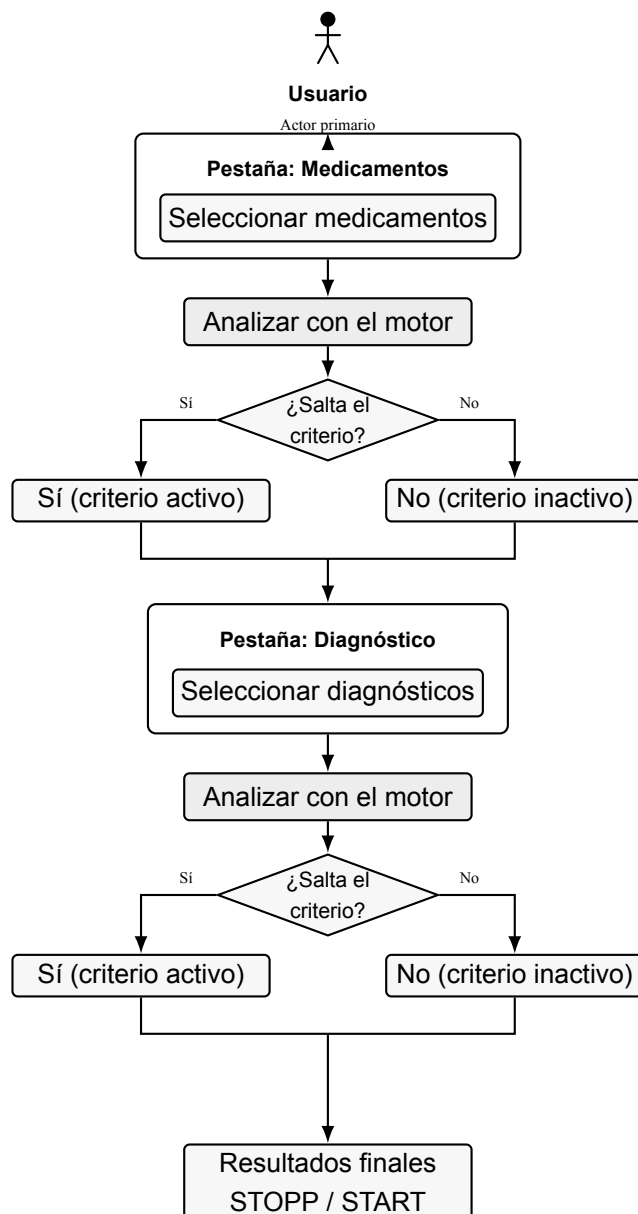


Figura K.1: Funcionamiento general de la solución: en cada pestaña el motor evalúa el caso y marca el criterio como activo o inactivo; el resultado final distingue avisos STOPP y START.

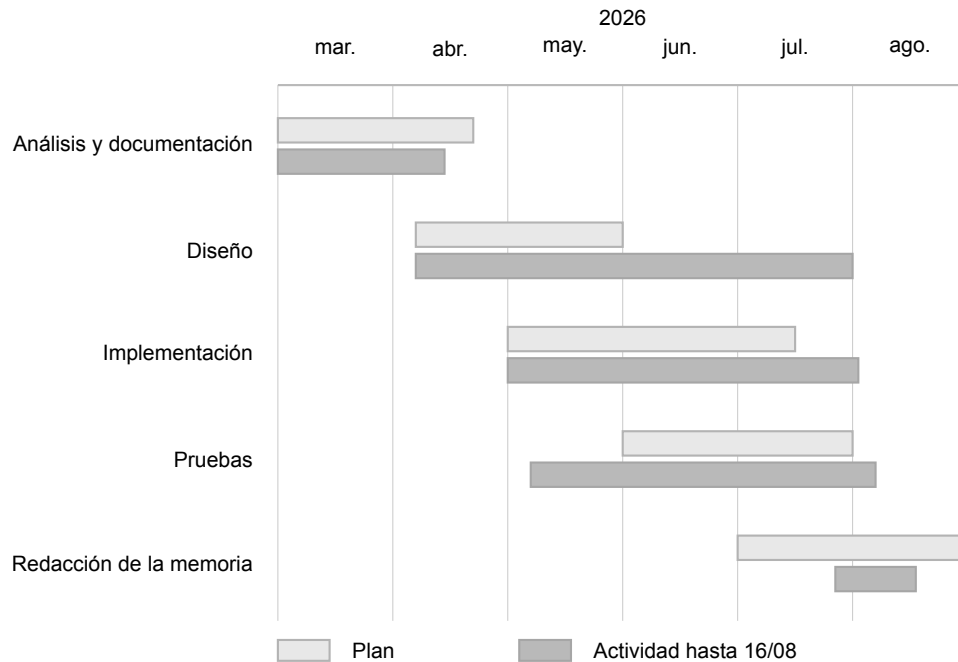


Figura K.2: Cronograma del proyecto (marzo–agosto de 2026): barras claras, planificación inicial; barras oscuras, actividad reconstruida mediante Git hasta el 16 de agosto. La fase de pruebas se solapa con la implementación. La redacción no se presenta como cerrada en esa fecha.

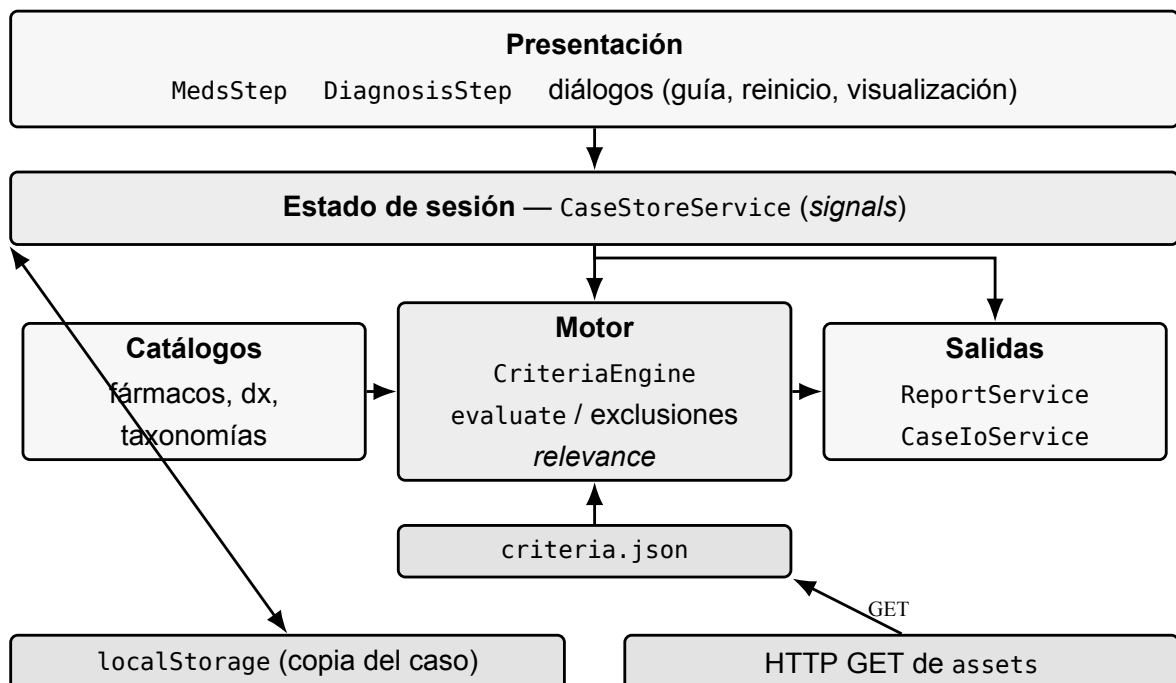


Figura K.3: Arquitectura de cajas: la UI actualiza el estado de sesión; el motor evalúa el caso con catálogos y `criteria.json`; las salidas leen una instantánea. El estado vive en memoria; `localStorage` guarda una copia en el navegador, no en una base de datos. `HttpClient` obtiene los activos mediante HTTP GET.

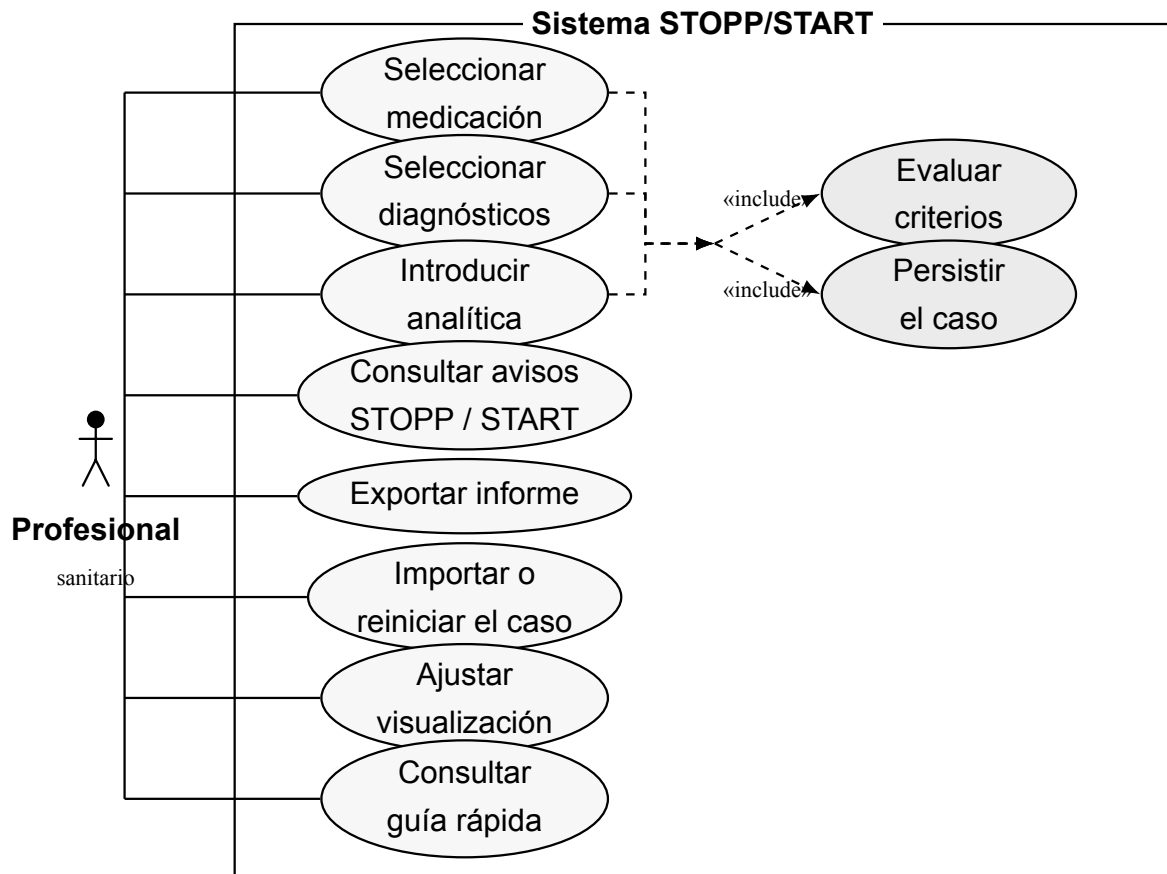


Figura K.4: Diagrama de casos de uso. Las líneas continuas asocian al actor con cada objetivo. Las discontinuas marcan la relación *include*: evaluar y persistir se ejecutan siempre que cambia la captura, sin invocación aparte.